Aerospace Science and Technology 176 (2026) 111868

Contents lists available at ScienceDirect

Aerospace Science and Technology

journal homepage: www.elsevier.com/locate/aescte

# HAURRT*smart: A rapidly exploring pseudo-random tree algorithm with low redundancy

Peidong He [a], Gang Hu [a,b,*], Essam H. Houssein [c,d], Guo Wei [e]

[a] Department of Applied Mathematics, Xi'an University of Technology, Xi'an, 710054, PR China
[b] School of Computer Science and Engineering, Xi'an University of Technology, Xi'an, 710048, PR China
[c] Faculty of Computers and Information, Minia University, Minia, Egypt
[d] Minia National University, Minia, Egypt
[e] University of North Carolina at Pembroke, Pembroke, 28372, NC, USA

ARTICLE INFO

Communicated by Dr Y Yanxing

*Keywords:*
Autonomous robot
Path planning technology
Halton sequences
Node redundancy
Target bias probability
Local branch pruning and reconnection
Dynamic environment

ABSTRACT

Autonomous robot path planning technologies have several problems. They can be slow, often use too many nodes, and struggle to adapt to complex environments, especially when using random sampling algorithms. Traditional Rapidly-exploring Random Tree (RRT) algorithms rely on random sampling and fixed stride lengths. This limits their ability to navigate complicated spaces and narrow passages effectively. This paper proposes the HAURRT*smart algorithm, which features four key improvements. First, we use Halton sequences for uniform sampling, eliminating node redundancy and step size dependence. By adding a state marker, our sampling strategy fully inherits the three major advantages of Halton sequences. Second, we add a dynamically adjusted target bias probability to balance exploration and exploitation for faster convergence. Third, the uniform node chain extension strategy addresses sparsity from long-distance node expansion, promoting compact connections. Finally, local branch pruning and reconnection, guided by the triangle inequality, optimize the final path and improve smoothness by eliminating redundant nodes in reverse order. We extensively test the HAURRT*smart algorithm on 10 pixel maps, ranging from simple to highly complex environments. The result shows that this algorithm has several key features: it can quickly find effective paths; it maintains low redundancy by exploring most areas with fewer nodes; it remains efficient and low in redundancy across different environments, showing strong adaptability; and it can also work in dynamic environments. Notably, at the reasonable setting of scene resolution, HAURRT*smart can rapidly explore all accessible region pixel blocks, a feat that surpasses all algorithms relying on regular random sampling. URL for supplementary materials and code: hepeidong.com.

## 1. Introduction

Autonomous robots are becoming more common in various industries and daily life due to advancements in technology [1]. They enhance productivity and provide conveniences [2]. A key challenge in robot autonomy is path planning, which involves finding a viable trajectory from the starting point to the target while avoiding obstacles [2]. Path planning technology is crucial for various applications, such as autonomous navigation in self-driving cars [3,4], agriculture [5], industrial processing [6], intelligent logistics [7], and military operations [8]. We can reduce drag and enhance operational efficiency by aerodynamically optimizing the intelligent body [9]. Meanwhile, high-performance path planning algorithms enhance operational efficiency and are essential for task execution [10]. Research in this realm must address environmental complexity and the need for real-time adjustments due to dynamic obstacles [11]. By integrating diverse technical approaches, researchers can develop safe and efficient movement strategies for robots in changing environments and requirements.

Path planning algorithms help find efficient paths in various environments and can be categorized into several types: graph search-based, biomimetic, learning-based, optimal control-based, and sampling-based algorithms [12], etc.

Graph-based path planning algorithms usually depend on a grid-mapped representation of the environment. Once the map is acquired, these algorithms generate paths by examining the connections between different grid regions. Those algorithms guarantee finding the optimal path after a finite number of searches and have high search efficiency [13]. Notable examples include Dijkstra [14], A* [15], and D* [16]. However, these algorithms struggle in high-dimensional spaces and complex environments, as the quality of their solutions relies on the

* Corresponding author.
*E-mail address:* hugang@xaut.edu.cn (G. Hu).

https://doi.org/10.1016/j.ast.2026.111868
Received 26 November 2025; Received in revised form 25 January 2026; Accepted 6 February 2026
Available online 14 March 2026

resolution of environmental discretization [17]. Higher resolution increases storage and computation demands exponentially, making them less effective in large or high-dimensional scenarios. So they are best suited for simple, small-map situations [18].

Biomimetic path planning algorithms are divided into neural network algorithms and traditional biomimetic algorithms [19]. Neural networks excel in nonlinear mapping, adaptability, and self-learning but suffer from long training times and a tendency to get stuck in local minima [19]. In contrast, traditional algorithms are more robust but require frequent adjustments to model parameters for optimal performance. In recent years, many heuristic algorithms have gained popularity for path generation [20,21]. Yu et al. enhanced the GWO algorithm with a DE ranking mutation strategy, proposing the HGWODE algorithm that can effectively tackle the problem of UAV path planning in complex environments [22]. Sarkar et al. introduced a domain-knowledge-based GA for mobile robots. This algorithm improves the planning performance of mobile robots by designing new operators, thus further boosting the planning efficiency and path quality [23]. Li et al. developed the IACO-IABC algorithm to optimize path length and reduce turns [24]. Various bio-inspired algorithms have also been modified for practical path planning applications [25–27], which contributes to the improvement of path length, safety, and smoothness. These algorithms are simple, flexible, independent of gradient information and objective functions, and capable of escaping local optima, offering robust global optimization [28,29]. However, they often require fine-tuning of adjustable parameters for different tasks to achieve optimal results [30]. Specifically, such algorithms converge to a single point, making them unable to capture surrounding data and address dynamic optimization problems [31].

Learning-based path planning is a significant advancement in automated navigation [32]. These methods optimize strategies through interaction with dynamic environments, enhancing perception and tackling complex challenges, demonstrating substantial advantages in sequential decision-making under uncertainty [33]. These algorithms often integrate reinforcement learning with specific scenario information to create tailored solutions [34,35]. Xu et al. employed an enhanced Deep Q-Network to optimize cinema evacuation paths, enabling rapid exploration [34]. Song et al. combined reinforcement learning with encounter models to meet maritime regulations for unmanned surface vehicles [36]. Wang et al. integrated robust adversarial reinforcement learning with curriculum learning to enhance the robustness of unmanned combat aerial vehicle trajectory planning [37]. Wang et al. proposed the E-MATD3 algorithm (combining evolutionary algorithms with multi-agent reinforcement learning) to address complex collaborative decision-making in multi-UAV aerial combat [38]. Zhao et al. enhanced the flexibility of UAV swarms and obstacle avoidance through graph neural networks and multi-agent reinforcement learning [39]. Additionally, integrating deep learning with traditional methods boosts performance. Song et al.'s KinoRRT* utilizes an improved TD3 reinforcement learning approach to guide RRT sampling, rapidly generating optimal paths that satisfy the kinematic constraints of unmanned underwater vehicles [40]. Jin optimized path planning for construction robots by integrating Q-learning reinforcement learning with artificial potential fields to avoid local optima and improve efficiency [41]. Sun et al. employed PointNet++ to construct heuristic regions, dynamically guiding RRT algorithm searches for efficient path planning [42].

Optimal control-based path planning algorithms start by establishing nonlinear motion equation models for mobile agents [43]. They tackle optimal control problems with path constraints. This approach enables the management of various constraints within a unified framework [44]. However, finding analytical solutions remains challenging due to the nonconvex constraints imposed by obstacles and nonlinear models [43]. Effective initialization strategies are crucial for converging to optimal local minima. Proper initialization strategies are needed for converging to good local optima [45]. Farid and Jungers proposed a binary combinatorial optimization algorithm. It reduces computational complexity for high-dimensional systems and addresses path planning and control optimization through dimensionality reduction [46]. Ren et al. developed an integrated strategy that maximizes longitudinal progress and minimizes lateral error [47]. They formulated velocity control and path tracking as a quadratic optimization problem to enhance adaptability in dynamic obstacle scenarios [47]. Zhou et al. introduced PMPPI, a model prediction path integral planner that runs multiple policy models in parallel to improve exploration and dynamic obstacle avoidance [48]. Wang et al. applied linear discretization techniques to transform satellite cluster trajectory planning into a convex optimization subproblem. This approach enhances both computational efficiency and collision avoidance reliability simultaneously [49]. Guo et al. developed a novel escape strategy for planetary rovers using SS gait and Bayesian optimization, improving adaptability and real-time performance in dynamic environments [50]. Qi et al. introduced a state-adaptive mode-switching controller. This enables more flexible and effective control of variable-track robots in complex terrain, thereby enhancing their adaptability [51]. The optimal control algorithm also enables merging globally planned paths with information from real-time perception systems. This allows the agent to dynamically avoid unexpected obstacles encountered during navigation [52].

Sampling-based path planning algorithms have garnered significant attention in recent years. These algorithms perform exceptionally in path planning for high-dimensional spaces, offering flexibility, scalability, probabilistic completeness, and adaptability to complex constraints [53]. Sampling-based methods have fewer parameters and a simple structure. They require minimal consideration of the map environment during execution, which results in higher search efficiency and faster search speeds. This makes them suitable for high-dimensional spaces [54]. When generating sampling points, they only verify the feasibility of the corresponding local path. This enables the rapid provision of conflict-free paths in the configuration space [53], thereby eliminating the need for spatial modeling [55]. Rapidly-RRT is a classic sampling-based path planning algorithm. It has attracted significant attention from the research community due to its high efficiency [56].

The RRT algorithm has several drawbacks, including slow updates, low execution efficiency, numerous redundant nodes, and limited adaptability to complex scenarios. We propose an improved version called HAURRT*smart. HAURRT*smart is high-efficient, has low redundancy, adapts well to complex environments, and can be used in dynamic environments. In the experimental part, it also showed its ability to cover all accessible areas rapidly.

## 2. Related works

This section is organized as follows: Section 2.1 defines the mathematical symbols used in path planning problems; Section 2.2 discusses recent advancements in the RRT algorithm, highlighting its advantages and disadvantages; Section 2.3 reviews research on the Halton sequence.

### 2.1. Mathematical definitions

Before introducing the relevant work, we provide definitions of mathematical symbols related to the problem. These mathematical symbols are consistent throughout this text. To assist readers in understanding the proposed methods, Table 1 lists the most commonly used symbols in the text along with their meanings.

By sampling and connecting nodes in the complete planning space, the sample-based path planning algorithm determines the final path. The complete planning space is defined as $M$. Then we have:

$$M_{free} \in M, \tag{1}$$

$$M_{obs} \in M, \tag{2}$$

$$M_{free} = M \setminus M_{obs}, \tag{3}$$

where $M_{free}$ denotes the reachable area in $M$, $M_{obs}$ denotes the unreachable obstacle area in $M$.

**Table 1**
The symbols with their Descriptions.

| Name | Description |
|---|---|
| $M$ | The entire area for path planning tasks, including both accessible and inaccessible areas. |
| $M_{free}$ | Accessible areas within the entire task area. |
| $M_{obs}$ | Inaccessible areas within the entire task area. |
| $S$ | A path, composed by path nodes. |
| $S_{init}$ | The start point of the path planning task. |
| $S_{goal}$ | The end point of the path planning task. |
| $\sigma(t)$ | Path node in the path, where $t$ is the index of the path node. |
| $L(S)$ | It is a function, calculating the total length of path $S$. |
| $R$ | The acceptance radius for the end point. The planning task ends when a tree branch reaches the terminal region of radius $R$. |
| $MaxIt$ | $MaxIt$ limits iterations in iterative algorithms to prevent infinite loops. If no feasible path is found within $MaxIt$ iterations, the task is considered failed. Otherwise, the remaining iterations optimize any found path. |
| $Tree$ | Random tree. |
| $Tree.nodes$ | Coordinates of path nodes in random tree. |
| $Tree.\ nearest$ | Parent node of the path node in random tree. |
| $Shoots$ | Part of the random tree. |
| $Shoots.nodes$ | Coordinates of path nodes in part of the random tree. |
| $Shoots.\ nearest$ | Parent node of the path node in part of the random tree. |
| $P_{rand}$ | The points generated by the sampling strategy may differ from the new node positions in the random tree. |
| $P_{nearest}$ | The nearest node to the new node among all nodes in the random tree. |
| $P_{new}$ | The actual new path node may differ from $P_{rand}$. |
| $P_{unbias\_rand}$ | By generating sampling points with a certain probability, algorithm convergence can be accelerated. |
| $Length(P_{new})$ | It is a function, calculating the total length from $S_{init}$ to the point $P_{new}$. |
| $Hs$ | A pre-generated set of pseudo-random sampling points. |
| $U$ | A set of flags indicating whether the point has been sampled, corresponding to $Hs$. |

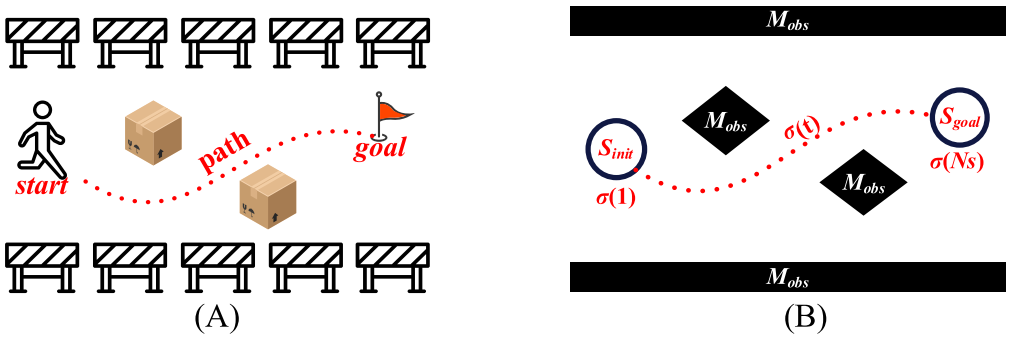



**Fig. 1.** Schematic illustration explaining certain mathematical symbols.

The sampling-based path planning algorithm aims to find a path from the start point to the goal point that meets specific requirements. This path is represented as $S = \{\sigma(t)|t = 1, 2, \ldots, Ns\}$, consisting of an ordered set of path nodes. Then we have:

$$\sigma(t) \in M_{free}, t = 1, 2, \ldots, Ns, \tag{4}$$

where $\sigma(t)$ denotes the $t$th path node, and $Ns$ denotes the quantity of path nodes.

Fig. 1 illustrates some mathematical symbols used in planning space and paths. Fig. 1(A) depicts an agent navigating from the starting point to the end point while avoiding obstacles, and Fig. 1(B) shows the corresponding mathematical symbols.

The goal of a path planning algorithm is to find a feasible path $S$ within a limited time while minimizing its length. Thus, the path planning problem can be defined as:

$$\textbf{\textit{Find:}}\ \text{Valid}\ \S = \{\sigma(t) \mid t = 1, 2, \ldots, Ns\}\ \text{with the smallest}\ L(S) \tag{5}$$

$$\textbf{\textit{s.t.}} \begin{cases} \sigma(t) \in M_{\text{free}},\ t = 1, 2, \ldots, Ns \\ \sigma(1) = S_{\text{init}} \\ \left\| \sigma(Ns - 1) - S_{\text{goal}} \right\| \le R \\ \sigma(Ns) = S_{\text{goal}} \\ \lambda \cdot \sigma(k) + (1 - \lambda) \cdot \sigma(k + 1) \in M_{\text{free}},\ 0 \le \lambda \le 1,\ k = 1, 2, \ldots, Ns - 1 \end{cases} \tag{6}$$

where $L(\ )$ is the length function, $S = \{\sigma(t)|t = 1, 2, \ldots, Ns\}$ is a feasible path, $L(S)$ is the length of path $S$, $S_{init}$ is the start point, $S_{goal}$ is the goal point, parameter $\lambda$ helps make sure that two path nodes can reach each other, $Ns$ is the quantity of path nodes, and $R$ is the acceptance radius for the end point.

Herein, the length value $L(S)$ for path $S$ is defined as the sum of the distances between the path nodes that make up path $S$. Then we have:

$$L(S) = \sum_{t=1}^{Ns-1} \|\sigma(t + 1) - \sigma(t)\|, \sigma(t) \in S, t = 1, 2, \ldots, Ns, \tag{7}$$

where $Ns$ denotes the quantity of path nodes of $S$.

### 2.2. Research on RRT algorithm

The RRT algorithm is a sampling-based path planning method for robot navigation in high-dimensional spaces [57]. The algorithm quickly explores feasible areas using random sampling to construct a random tree, where nodes represent path points and edges represent feasible connections. As the tree grows by extending to new sampled points, the algorithm progresses toward the target. It then terminates when a certain node is close to the target. Notably, the RRT algorithm features low complexity and quickly generates candidate paths. Given

enough iterations, it offers an approximate alternative to the optimal path, demonstrating probabilistic completeness.

**Theorem 1(Probabilistic Completeness):** Define event $\psi$ as the existence of an accessible path for path planning, and event $\chi$ as the RRT algorithm finding that path. Then we have:

$$\lim_{k\to\infty} P(\chi|\psi) = 1. \tag{8}$$

**Proof.** Suppose there exists a path $S$ such that $\sigma(1) = S_{init}$ and $\sigma(Ns) = S_{goal}$, and this path has a safety radius of width $\eta$. We may assume that $\|\sigma(k) - \sigma(k+1)\| \leq \eta$ for $k = 1, 2, \ldots, Ns-1$. Create a sphere with radius $r$ centred at each path node $\sigma(k)$, denoted as $V(k)$, such that $r < \eta/2$. At each point, any space within the sphere is reachable. Let the node $\sigma(i)$ in the random tree be denoted as $\varsigma_i$. Since the probability $P_{i,1} = (Area\ of\ ball\ V(k)/\ Total\ space\ area) > 0$ that the sampling point falls into $V(k)$, and the expansion success rate $P_{i,2} \approx 1$ of the sampling point $\sigma(i)$ to $\sigma(i+1)$, therefore $\mathrm{P}\left(\varsigma_{i+1}|\varsigma_i\right) > 0$. Therefore, $\lim_{k\to\infty} P(\chi|\psi) = 1$.

The RRT algorithm is more efficient and flexible than bio-inspired and graph theory-based path planning algorithms, with the advantage of probabilistic completeness. However, its reliance on random sampling leads to instability and uncertainty [58], especially in complex terrains, which can slow convergence or prevent viable path solutions [56]. While RRT is suited for scenarios requiring quick responses over precision, its instability and uncertainty often result in path solutions that diverge considerably from optimal outcomes [53]. Additionally, when selecting sampling methods for the RRT algorithm, it's crucial to balance exploration and exploitation [59]. Excessive exploration can slow convergence and create gaps between random tree branches, while insufficient exploration may lead to suboptimal paths and stagnation at local minima. If gaps occur at narrow passages, it results in low travers ability, indicated by a small $\eta$ value in Theorem 1, which affects the $V(k)$ areas and leads to a small $P_{i,1}$. Although the RRT algorithm can theoretically find a feasible path, doing so may require many iterations and may not achieve true completeness in finite time.

To address the existing shortcomings of RRT, researchers made necessary adjustments to the various execution structures of the algorithm, thereby increasing computational efficiency, reducing planning time, and mitigating several of its limitations.

Ganesan highlighted that the sampling process in the RRT algorithm significantly affects performance [59]. Traditional RRT algorithms pick points at random across the whole area, which can make building the search tree slow [59]. To address this, improved methods focus on selecting points within specific regions, each using a different strategy to balance speed and solution quality. For example, Informed RRT* accelerates finding solutions in complicated spaces by sampling inside an ellipsoid, reducing computation time [60]. Yu and Chen extended this to 3D with Cyl-IRRT* [61]. RRT*smart enhances speed further by focusing samples near key 'beacon' locations to find better routes faster [62], while AE-RRT* (Cao et al.) uses a changing elliptical region to pick more promising points, boosting both speed and path quality [63]. However, these focused approaches can sometimes cause the search to get stuck in one region and overlook alternative routes [64]. Li et al. observed that restricting the sampling region works well in simple environments, but can lower randomness and limit the quality and variety of solutions [65]. Jin et al. improved planning rates by 27.03% by adjusting random sampling distribution with a Gaussian function to increase sampling near obstacles [66]. Zhang et al. implemented a biased sampling strategy to enhance exploration efficiency by rejecting nodes outside a specific interval and discarding high-cost paths [67]. Huang reduced the time to obtain initial solutions with dynamic gradient sampling and optimized paths by reconstructing outputs [68]. Choosing sampling strategies based on probability helps maintain both efficiency and solution quality [13].

Low-variance sampling is a technique that enhances the global search efficiency of the RRT algorithm [56]. The fundamental principle of low-variance sampling is to minimize sampling randomness, resulting in a more uniform distribution across the search space. This approach promotes more effective exploration and accelerates the path planning process. Research by Zhong et al. showed that uniformly distributed sampling points enhance the exploration capabilities and convergence speed of the RRT algorithm in complex environments [69]. Additionally, Tahmasbi et al. used kd-tree partitioning to divide the maps into regions, with Q-learning serving as the high-level decision-maker [70]. Their focus on addressing regional connectivity issues aimed to improve both temporal efficiency and success rates in high-dimensional path planning [70]. Zhang and Duan introduced an innovative search method based on variational-resolution probabilistic graphs with alternative grid configurations to support global searches for agents with variable phases and behaviors [71]. Cui et al. also contributed by introducing a sparse sampling mechanism into MQ-RRT*, optimizing the selection of sampling points [13]. When combined with a dynamic target bias strategy and a novel parent node creation method, this approach significantly enhanced both the quality of initial paths and convergence speed. These examples illustrate that low-bias sampling strategies, particularly when integrated with other enhancements, can substantially improve the efficiency of RRT algorithms across a variety of environments.

An et al. noted that varying sampling step sizes during different search stages can enhance initial path generation time and algorithm efficiency [72]. Traditional path planning algorithms typically use a fixed step size: a small step size slows convergence, whereas a large one can cause collisions and hinder node expansion [73]. Adaptive step size adjustments are crucial for improving convergence speed. Feng et al. introduced DBVS-APF-RRT* with a variable step size to boost path generation speed in complex environments [74]. Liu et al. also utilized an adaptive step size for ERRT* [75]. Building further, RRT-D* adopts an adaptive step strategy to enhance efficiency. However, unlike DBVS-APF-RRT* and ERRT*, it requires environmental grid processing, increasing computational complexity [54].

Sampling-based path planning algorithms are widely used in embedded systems because they offer speed, scalability, and low complexity. In memory-limited systems, reducing memory usage is crucial [76], and this mainly depends on the number of nodes. Increasing the number of nodes improves path search accuracy but also raises memory requirements due to redundancy. Conversely, using fewer nodes can miss important areas along the optimal path, leading to suboptimal solutions. Therefore, implementing RRT in memory-constrained systems requires balancing node redundancy [65]. An environment preprocessing strategy can make sampled nodes more effective and reduce redundancy, but it may lower initial planning efficiency [77]. Jiang et al. proposed the R2-RRT algorithm using the SMR program for generating probabilistic mobile graphs, enhancing path generation reliability [78]. Ning et al. improved the RRT algorithm exploration in complex environments by employing Voronoi diagrams [79]. Kim et al. optimized path quality with generalized Voronoi diagrams for sampling according to Cloud RRT* [80]. Liu et al. accelerated initial solution acquisition through convex decomposition and delayed rewiring in high-dimensional spaces according to CDRT-RRT* [81]. Uwacu et al. enhanced path planning efficiency in narrow channels using hierarchical skeleton-guided expansion [82]. Meng et al. embedded risk contour maps into deterministic feature maps to speed up convergence [83]. While environmental preprocessing improves planning speed and reduces node redundancy, Guo et al. cautioned that it can sacrifice path quality and the efficiency of initial path generation. It also lacks flexibility when dealing with dynamic planning spaces [77]. In practical applications, allowing algorithms to output approximate alternative solutions for optimal path schemes can reduce node redundancy and alleviate memory pressure [76].

The RRT algorithm faces issues with invalid tree expansion, which can also lead to node redundancy. For example, if A, B, and C are three consecutive path nodes in the planning environment (with B and C being close to each other), expanding the branch from C to B is invalid given that nodes A and C already exist. Yu et al. suggest that effective pruning and reconnecting branch structures can address this issue

during the expansion process [56]. Huang et al. enhanced algorithm success rates in narrow channels. They achieved this by adding repulsive fields during node expansion and reducing tree nodes [84]. Fan et al. integrated improved artificial potential fields to guide new node generation with attractive and repulsive forces, avoiding local minima issues [85]. Chen and Wang optimized parent node selection for faster convergence according to TQRRT* [86], and Li et al. expanded parent node choices to include neighboring nodes, improving search performance despite requiring parameter adjustments [65]. Zhang et al. used a rewiring strategy to connect ancestor and parent nodes, boosting efficiency [87]. Huang et al. optimized paths with a reconnection mechanism to leverage previous samples [88]. Li et al. focused on underwater glider path planning research. They used sea current impact assessments to guide parent node selection. This effectively addresses the impact of sea current changes on optimal path solutions in complex marine environments [89]. Yao et al. adjusted node expansion using probabilistic factors, enhancing path search efficiency [90]. Overall, these studies demonstrate a key finding: modifying node expansion methods can significantly improve path planning algorithm performance.

Path smoothing is a key step in path planning. It reduces sudden speed changes, which helps the agent move more smoothly and efficiently [91]. The RRT algorithm produces continuous path nodes that often contain redundant sampling points and lack smoothness, making them unsuitable for real-world applications. Subsequent processing is needed for RRT-type outputs [53]. Alongside pruning and reconnection strategies, curve smoothing techniques are used to enhance path smoothness and safety. Wang et al. used an improved Bézier curve algorithm to smooth paths, reducing energy consumption and enhancing safety [92]. Wang et al. also applied linearization and curve fitting for post-processing, ensuring geometric continuity and practicality [93]. Xu et al. adjusted path nodes via gradient descent for safe distances, using B-splines and cubic splines for smoothing [94]. Yu et al. enhanced node expansion with an improved Steering function and Hermite interpolation methods to maintain smoothness [95]. By modifying the cost function of RRT-class algorithms, various complex requirements for paths can also be balanced. Lindqvist et al. integrated information gain into RRT-class algorithms. This integration optimizes exploration in narrow spaces [96]. Zhang et al. modified the cost function of random trees to balance path length and risk [97]. Liang and Zhao developed CCPF-RRT* to consider congestion intensity and path length, effectively navigating complex urban environments [10].

The contributions of this paper are as follows:

**(1) We propose the ordered sampling node sequence strategy:** This strategy directly controls the position of path sampling nodes using the Halton sequence, enhances sampling node uniformity, reduces node redundancy, improves algorithm efficiency and eliminates sensitivity to step size.

This strategy is the core innovation of this paper. It differs from the indirectly pseudo-random-based sampling strategies in [69,98,99] discussed in Section 2.3. First, this strategy uses a pseudo-random distribution to directly control node distribution in the random tree, instead of only guiding the sampling direction. Compared to multi-resolution uniform-grid sampling, this method eliminates the need to preprocess the planning space, thereby improving algorithm efficiency. Second, it adds a state flag to the pseudo-random sequence, which helps maintain uniform exploration across the searchable space. Third, this state flag also allows the strategy to be extended for use in dynamic environments. This strategy alone balances exploration and exploitation, adjusts step sizes, and adapts to environmental conditions.

**(2) We integrated existing technologies to fix the shortcomings caused by Contribution 1.**

We propose an adaptive target bias strategy. This approach adjusts the bias probability based on the positions of sampled points, helping balance search efficiency and exploration. We also introduce a uniform node chain extension strategy. This method promotes compact connections between nodes and enables the lateral growth of long-distance random-branch trees. Additionally, we borrow a path optimization strategy from RRT*smart. This technique reduces node redundancy in the final path and improves path smoothness.

**Algorithm 1:** RRT.

**Input:** $M$(include $M_{\text{free}}$ and $M_{obs}$), $S_{init}, S_{goal}, R, MaxIt$
**Output:** $S = \{\sigma(t) \mid t = 1, 2, \ldots, Ns\}$
$Tree.nodes = \{S_{init}\}$, $Tree.nearest = \{Nan\}$; # { Nan } is an empty placeholder element
**while** $t < MaxIt$ **do**
  $P_{rand} \leftarrow random\_sampling(M)$;
  $P_{nearest} \leftarrow get\_nearest\_node(P_{rand}, Tree.nodes)$;
  $P_{new} \leftarrow get\_extended\_node(P_{rand}, P_{nearest})$;
  **if** $path\_feasible(P_{new}, P_{nearest})$ **then**
    $Tree.nodes \leftarrow \{Tree.nodes, P_{new}\}$;
    $Tree.nearest \leftarrow \{Tree.nearest, P_{nearest}\}$;
  **end if**
  **if** $\left\|P_{new} - S_{goal}\right\| \leq R$ **then**
    $Tree.nodes \leftarrow \{Tree.nodes, S_{goal}\}$;
    $Tree.nearest \leftarrow \{Tree.nearest, P_{new}\}$;
    **break**;
  **end if**
  $t = t + 1$;
**end while**
$S \leftarrow get\_path(Tree, S_{goal})$;

Algorithm 1 provides pseudocode for the basic RRT algorithm.

The algorithm flow components used in Algorithm 1 are described as follows:

$random\_sampling(M)$: Given a planning space $M$, this flow component returns a random point within the space $M$. In the traditional RRT, this point is a randomly selected point from $M$.

$get\_nearest\_node(P_{rand}, Tree.nodes)$: Given a point $P_{rand}$ and a set of path nodes $Tree.nodes$, this flow component returns the node in $Tree.nodes$ that is closest to $P_{rand}$.

$get\_extended\_node(P_{rand}, P_{nearest})$: Given a point $P_{rand}$ and a path node $P_{nearest}$, this process component returns a point extended from $P_{nearest}$ to $P_{rand}$ by a distance of $step\_size$, which must be strictly restricted to the defined range of the space $M$.

$path\_feasible(P_{new}, P_{nearest})$: Given two points $P_{new}$ and $P_{nearest}$, this process component is used to detect the connectivity of the line segment between $P_{new}$ and $P_{nearest}$ in $M$. If there are obstacles, it returns false; otherwise, it returns true.

$get\_path(Tree, S_{goal})$: Given a task goal point $S_{goal}$, this process component returns a connected path $S$ based on the $Tree.nodes$ and $Tree.nearest$ of the random tree $Tree$. If $S_{goal}$ is not included in $Tree.nodes$, it indicates that the RRT algorithm did not find a feasible path before reaching the maximum iteration count $MaxIt$, and this process component returns an empty path.

### 2.3. Research on Halton sequence

The Halton sequence is a deterministic low-discrepancy sequence for generating uniformly distributed pseudo-random points in multidimensional space [100]. It relies on the radical inverse function to project an integer $n$ onto the interval $[0, 1)$ for generating the sequence. The expression for the radical inverse function is:

$$\phi_b(n) = \sum_{i=0}^{k} a_i(n, b) b^{-i-1}, \tag{9}$$

where $b$ is a prime base, $n$ is the sequence index, $a_i(n, b)$ are the digits of $n$ in base $b$, and $k$ is the largest integer such that $b^k \leq n \leq b^{k+1}$.

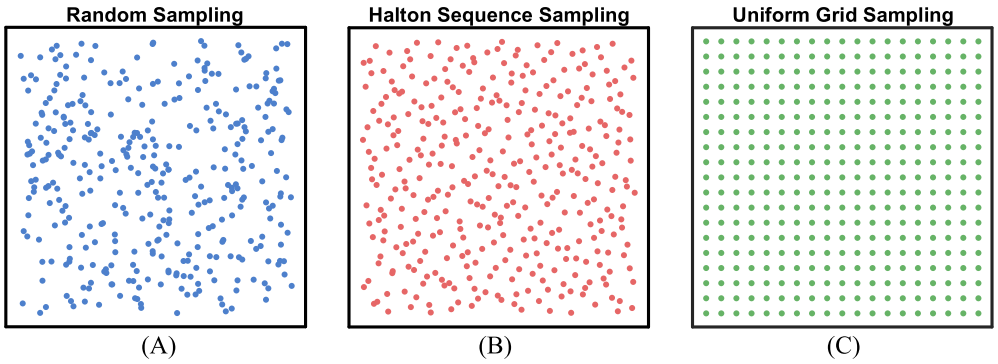



**Fig. 2.** Schematic description of the 3 sampling methods.

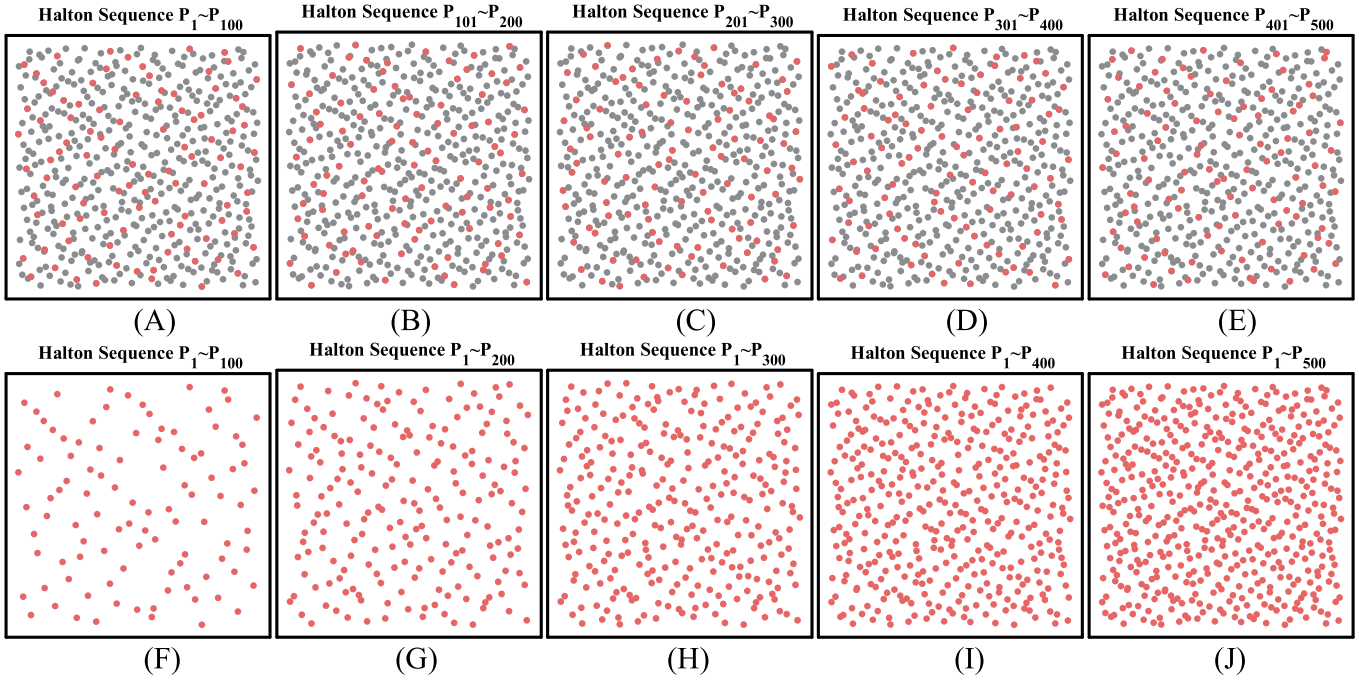



**Fig. 3.** Position maps of pseudo-random sampling points in different sequence intervals.

The Halton sequence enhances spatial coverage by minimizing clustering and filling structural gaps found in pseudo-random sampling. Fig. 2 illustrates three sampling methods in space.

The Halton sequence has three main features that explain why it is used. The Halton sequence demonstrates dimensional independence due to its prime base $b$, allowing for independent pseudorandom sequences for different bases. For instance, in Fig. 2(B), the horizontal and vertical directions use bases 3 and 5, respectively. It also exhibits continuous low variability. Fig. 3(A-E) shows 500 consecutive Halton points divided into five groups, where red points indicate the current group, while grey points represent the other 400 points outside of these 100. Fig. 3(A-E) highlights that points maintain good uniformity within any interval. Additionally, the Halton sequence shows progressive uniformity, as demonstrated in Fig. 3(F-J), where adding 100 points from an adjacent interval always maintains relatively uniform spacing with existing points.

The Halton sequence shares key characteristics with the RRT sampling points. Establishing a one-to-one correspondence could potentially enhance its performance. Its advantages include: (1) Dimension independence for high-dimensional planning; (2) An ordered sequence that enables adaptive pre-processing of the planning space through discretization, without increasing additional time complexity; (3) Continuous low variance, which maintains randomness in sampling points over iterations; (4) Progressive uniformity, allowing for gradual filling of unsampled areas and reduced node redundancy by adaptively adjusting step sizes.

Researchers have noted the connection between Halton sequences and sampling points in RRT algorithms. Zhong et al. proposed the HB-RRT algorithm, which uses the Halton sequence for uniformly distributed, low-redundancy sampling in GPS-denied environments [69]. Zhong et al. enhanced the HBC-RRT's exploration by replacing random sampling with Halton sequences [99]. Huang et al. used the Halton sequence to generate uniformly distributed samples, improving H-IABFMT* efficiency in tea tree canopies [98]. However, these algorithms do not fully utilize the Halton sequence's advantages. In contrast, the HAURRT*smart algorithm uses labeled sampling nodes. It tracks whether points from the Halton sequence have been sampled. This labeling enables the algorithm to resample pseudo-random path nodes, enabling it to achieve the highest coverage (Measured at the appropriate pixel precision) in accessible areas under most experimental scenarios. As a result, the algorithm demonstrates probabilistic completeness quickly and increases execution efficiency with minimal redundancy

among sampled nodes, as demonstrated by experimental results. The state marker also allows HAURRT*smart to handle dynamic environments effectively, as experimental results confirm.

## 3. HAURRT*smart

The analysis in Section 2.2 shows that merely increasing node sampling density does not effectively improve RRT algorithm performance and can lead to redundant points. Instead, enhancing the quality of sampling points is crucial. We modified the *random_sampling* and *get_extended_node* components in the RRT algorithm for better control over node generation. Additionally, we introduced an Adaptive target bias strategy to the HAURRT*smart algorithm to adjust weights for faster convergence and exploration of the planning environment. We removed the *step_size* parameter in *get_extended_node*, which could create long branch structures. To address this, we added a uniform node chain extension strategy to limit maximum branch length. Further, we implemented a path optimization strategy to improve path smoothness and reduce node redundancy.

### 3.1. Ordered sampling node sequence based on Halton sequence strategy.

We utilize the Halton sequence to optimize the position of the new node $P_{new}$ generated by the *get_extended_node* component. During initialization, we define an ordered Halton sequence sampling point set, $Hs(n)$, containing $\infty$ sampling points. The expression for $Hs(n)$ is:

$$H_s(n) = \left\{ \left( \phi_{b_1}(n), \left( \phi_{b_2}(n) \right) \right) \mid n = 1, 2, \ldots, \infty \right\}, \tag{10}$$

where $b_1$ and $b_2$ denote two different prime bases, here $b_1 = 3$ and $b_2 = 5$. These two parameters are analogous to random number seeds and can be specified arbitrarily.

Furthermore, we defined a sequence $U(n)$ to record whether the path nodes in $Hs(n)$ were sampled, and $U(n)$ is expressed as:

$$U(n) = \begin{cases} 1, & H_s(n) \text{ has already been sampled} \\ 0, & \text{else} \end{cases}, \tag{11}$$

where $n$ represents the index corresponding to the Halton sequence sampling point $Hs(n)$.

In this strategy, we adjust lines 3, 4, and 5 of Algorithm 1 to use $Hs$ and $U$ for determining optimal $P_{new}$ and $P_{nearest}$, as outlined in Algorithm 2. The parameter $st$ records the number of times $P_{new}$ failed to expand on the random tree in previous iterations, allowing HAURRT*smart to select path nodes at fixed positions on $Hs$. The function $get\_st\_sampling\_nodes(Hs, U, st)$ returns the $st$th $Hs(n)$ where $U(n) = 0$. Additionally, we introduce randomness to HAURRT*smart by skipping some sampling points. As indicated in the opening line of Algorithm 2, the parameter $st$ increases by a randomly selected value from 1 to 3. This technique is heuristic. With the inclusion of the label $U$, the sampling probability for positions with lower indices in $Hs$ gradually approaches 100% (which is evident and thus omitted). Consequently, this method can be adapted to particular needs. For instance, increasing the number of skipped items promotes greater randomness and exploration, whereas decreasing it prioritizes stability. If the first line of Algorithm 2 is not executed, the same planning problem will yield identical random trees due to the quasi-random Halton sequence used for generating path nodes $P_{new}$.

Fig. 4 illustrates the ordered sampling node sequence based on Halton sequence strategy. In Fig. 4, path nodes closer to the start are green, those closer to the endpoint are red, nodes within obstacles are blue, and extended nodes outside $Hs$ are yellow. In Fig. 4(A), the straight path to node 1 is extendable, offering an advantage over traditional RRT algorithms by not relying on the parameter *step_size*, though with a lower expansion rate due to its length. In Fig. 4(B), node 2 is can be expanded further ($U(2) = 0$). Fig. 4(C) shows node 4 not being expanded ($U(4) = 0$), while node 5 spawns a new branch. Node 6, within an obstacle, leads to the generation of an extended node near $P_{nearest}$, better capturing obstacle information. Fig. 4(D) depicts the random tree before expanding node 9, with no red nodes indicating an untouched obstacle region. Fig. 4(E) shows the first expansion of red path node 10, as the random tree circumvents the obstacle. Finally, Fig. 4(F) highlights nodes 2, 4, and 8 being prioritized for expansion to maintain balanced sampling density around the obstacle, enhancing convergence stability.

The Halton sequence's progressive uniformity allows for gradual space filling, as shown in Fig. 4(G, H, I). This feature enables HAURRT*smart to uniformly distribute path nodes in narrow channels, supporting quick and robust pathfinding in complex scenes. Fig. 5 provides a schematic diagram of how the Halton sequence, by progressively filling space, improves path generation in complex scenes; irrelevant paths have been removed for visual clarity.

**Algorithm 2:** *HAURRT*smart_get_sampling_node.*

**Input:** $M, Hs, U, st, Tree$
**Output:** $P_{new}$, $P_{nearest}$
1: $st = st + (r_1 < 0.5) + (r_2 < 0.5) + (r_3 < 0.5)$;
2: $P_{rand} \leftarrow get\_st\_sampling\_nodes(Hs, U, st)$;
3: $P_{nearest} \leftarrow get\_nearest\_node(P_{rand}, Tree.nodes)$;
4: **if** $P_{rand} \in M_{free}$ **then**
5: $P_{new} \leftarrow P_{rand}$;
6: **else**
7: $P_{new} \leftarrow P_{nearest} + step\_size \bullet (P_{new} - P_{nearest}) / \left\| P_{new} - P_{nearest} \right\|$;
8: $U(z) \leftarrow 1$;
9: **end if**

### 3.2. Adaptive target bias strategy

Target bias strategies improve RRT algorithms by guiding paths more efficiently toward the target, increasing convergence speed.

Algorithm 3 outlines the adaptive target bias strategy, which enhances search efficiency while maintaining the RRT algorithm's progressive optimality. In the early stages of random tree generation, as shown in Fig. 4(H), the Halton sequence path sampling points near the target have a large distance gap. This results in $Br$ maintaining a high value, which boosts the sampling probability of the target point and accelerates the tree's growth towards it. Once the tree covers the planning space $M$, the Halton sequence's uniformity helps fill gaps between path nodes, focusing the algorithm on minimizing path length and balancing the value of $Br$. This innovation allows HAURRT*smart to quickly generate smooth, short paths in complex environments through probabilistic goal-oriented sampling.

Traditional unbiased sampling enhances convergence by guiding the tree's growth. However, this effect is not practical in HAURRT*smart due to the influence of a pseudo-random sequence on the distribution of path nodes, as the ordered sampling node sequence is generated using the Halton sequence. The tree's growth focuses on filling the task's reachable space. According to Algorithm 3, sampling points can be replaced with target points with a certain probability. For example, in Fig. 4 (E), if the target point replaces the next sampling point, the initial path can be generated directly. This is precisely why this approach accelerates algorithm convergence.

Regarding the 0.1 coefficient used to calculate the bias probability $Br$, it can be appropriately increased when there are fewer obstacles near the task target point or when the overall planning space is relatively simple or small. Conversely, this coefficient should be appropriately decreased under different circumstances. This strategy is executed only when the initial path is absent, minimizing its impact on the random tree distribution.

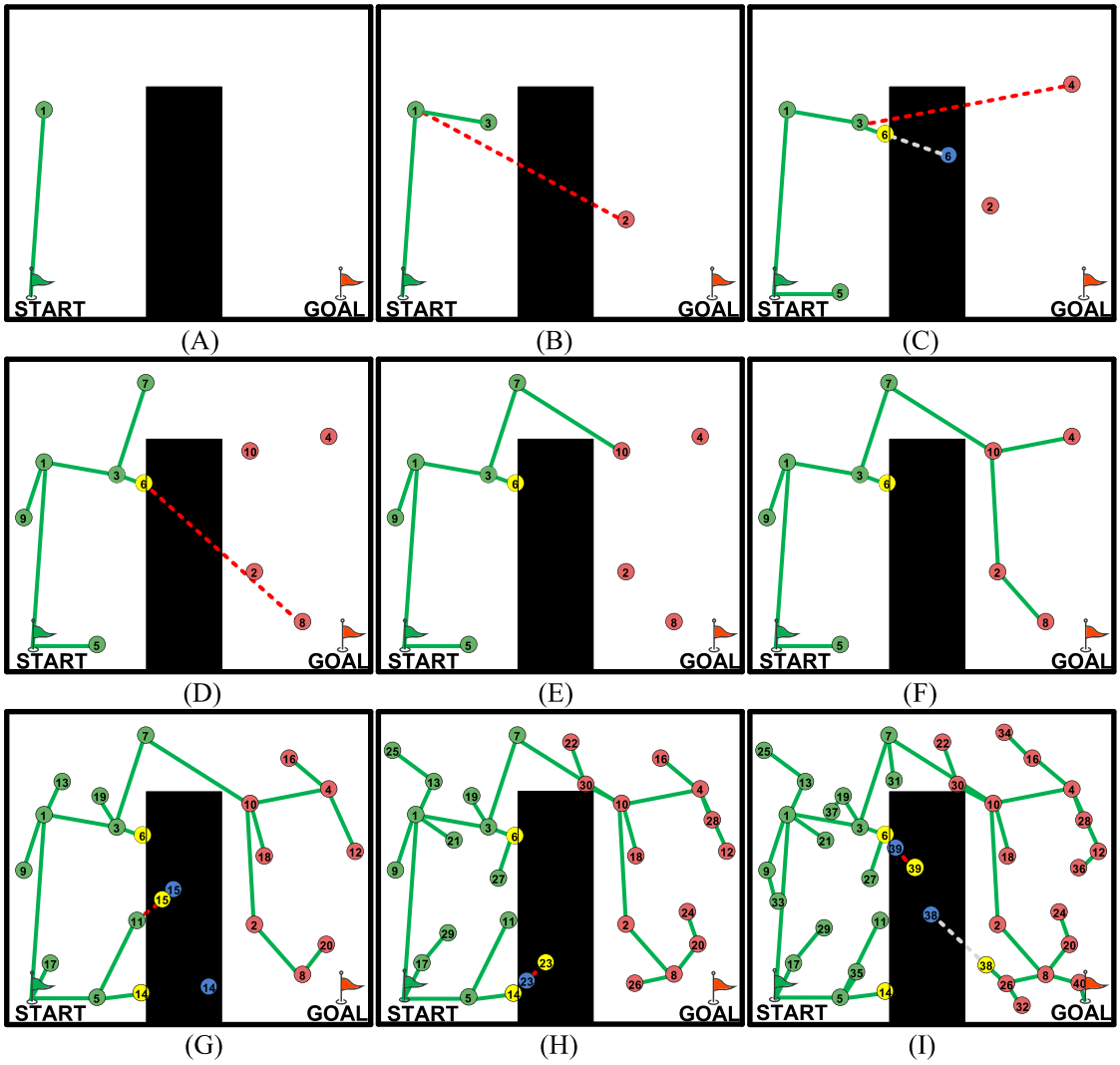



**Fig. 4.** Illustration of ordered sampling node sequence based on Halton sequence strategy.

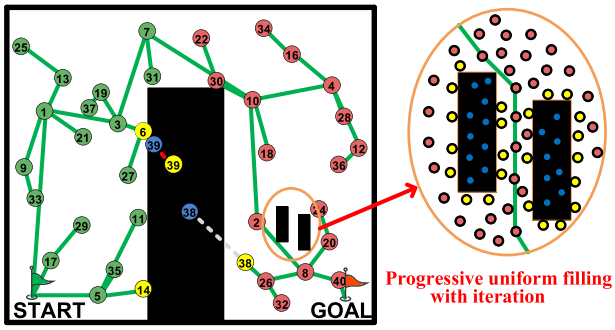



**Fig. 5.** Diagram illustrating the ability of the Halton sequence to generate paths.

**Algorithm 3:** *HAURRT*smart_adaptive_target_bias.*

**Input:** $P_{unbias\_rand}, S_{init}, S_{goal}$
**Output:** $P_{rand}, P_{nearest}$
1: Calculate the bias rate
$Br = 0.1 \times \left(1 - \left\|S_{goal} - P_{unbias\_rand}\right\| / \left\|S_{goal} - S_{init}\right\|\right)$;
2: **if** $r < Br$ and $Initial\ path\ not\ found$ **then**
3: $P_{rand} = S_{goal}$;
4: **else**
5: $P_{rand} = P_{unbias\_rand}$;
6: **end if**
7: $P_{nearest} \leftarrow get\_nearest\_node(P_{rand}, Tree.nodes)$;

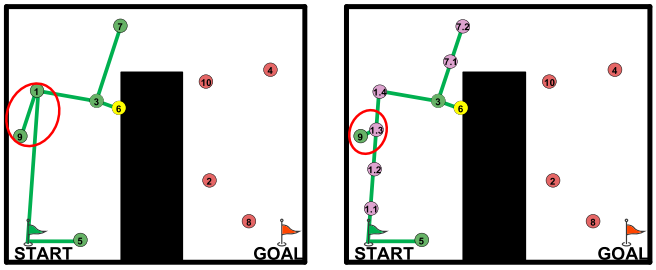
### 3.3. Uniform node chain extension strategy.

Controlling the distribution of path nodes using an ordered sampling sequence based on the Halton strategy can reduce reliance on the parameter *step_size* and improve convergence speed in RRT and related algorithms. However, drawbacks remain. For instance, in Fig. 4, some paths deviate from the ideal direction. Specifically, in Fig. 4(D), node 9's growth direction is opposite to the optimal clockwise arc from the start to the target. This issue arises because the distance between $P_{new}$ and $P_{nearest}$ in HAURRT*smart is adjusted based on Halton sampling density, leading to longer distances and insufficient sampling density among some nodes in early iterations. To address this, we enhanced the random tree expansion process for $P_{new}$ and $P_{nearest}$ in HAURRT*smart, detailed in Algorithm 4.

In Algorithm 4, during the expansion phase of $P_{new}$, the algorithm discretizes the path from $P_{new}$ to $P_{nearest}$ using the *step_size* and connects those nodes. The schematic in Fig. 6 illustrates this process, with Fig. 6(A) derived from Fig. 4(D). The expanded nodes in

**Algorithm 4:** *HAURRT*smart_extend_nodes.*

**Input:** $Tree, P_{new}, P_{nearest}$
**Output:** $Shoots$
Calculate the distance $D = \|P_{new} - P_{nearest}\|$;
Calculate the number of nodes $Nn = D/step\_size$, round up;
$i = 0$, $Shoots.nodes = \varnothing$, $Shoots.nearest = \varnothing$;
**while** $i \le Nn$ **do**
  $i = i + 1$;
  $Shoots.nodes \leftarrow \{Shoots.nodes, P_{nearest} + i \times (P_{new} - P_{nearest})/Nn\}$;
  $Shoots.nearest \leftarrow \{Shoots.nearest, P_{nearest} + (i-1) \times (P_{new} - P_{nearest})/Nn\}$;
**end while**



(A) (B)

**Fig. 6.** Diagram of the uniform node chain extension strategy.

HAURRT*smart form a chain, which we call the uniform node chain extension strategy.

Key strategic details need clarification. The uniform node chain extension strategy does not significantly affect algorithm complexity since long-distance nodes usually appear early in sampling and have low extension success rates due to obstacles. As iterations continue, the Halton sequence ensures that extended nodes are evenly distributed across reachable $M_{free}$ regions, gradually decreasing the distance from $P_{new}$ to $P_{nearest}$. Furthermore, this strategy does not imbalance the probability of node extensions, as its effect is limited to rare cases in the early sampling phase.

### 3.4. Path optimization strategy.

We use three algorithm components, *Choose_Parent*, *Rewire*, and *Optimise_path*, to optimise the generated random tree and its paths.

The *Choose_Parent* component functions after each sampling, as outlined in Algorithm 5. It uses $Length(\star)$ for the shortest path to node $\star$ and $R_{near}$ as the neighbourhood search radius. Once a node is added to the random tree, its path length to the sampling point is further optimized within $R_{near}$. If a neighboring node has a shorter path, it replaces $P_{nearest}$ as the parent of the current sampling node. Fig. 7(A, B, C, D) illustrate this process. In Fig. 7(A), the path from start to end is winding due to limited sampling points. In Fig. 7(B), a new node 4 is sampled and becomes a branch since it's closer to node 2. Then, in Fig. 7(C), node 4 searches for a new parent within $R_{near}$, determining that node 1 would shorten the path while maintaining reachability. Consequently, in Fig. 7(D), the path is adjusted to connect node 4 directly to node 1.

*Rewire* acts after the *Choose_Parent* component and affects on the same path node. Its pseudocode is shown in Algorithm 6.

The *Rewire* component's schematic is shown in Fig. 7(E, F). For this path node, the *Choose_Parent* process ensures the path from the starting point is optimal. The *Rewire* component optimizes parent nodes of nearby path nodes (within the $R_{near}$ radius range). In Fig. 7(E), for node 3, the path from node 4 to node 1 is shorter than the original. Thus, in Fig. 7(F), node3ŕs parent is updated to node 4. Comparing Fig. 7(A) and

**Algorithm 5:** *Choose_Parent.*

**Input:** $P_{new}, P_{nearest}, Tree$
**Output:** $P_{parent}$
$P_{parent} = P_{nearest}$;
$L_{\min} = Length(P_{nearest}) + \|P_{new} - P_{nearest}\|$;
**for** each $Tree.nodes(i)$ **do**
  **if** $\|P_{new} - Tree.nodes(i)\| < R_{near}$ **then**
    $L_{potential} = Length(Tree.nodes(i)) + \|P_{new} - Tree.nodes(i)\|$;
    **if** $path\_feasible(P_{new}, Tree.nodes(i))$ **then**
      **if** $L_{potential} < L_{\min}$ **then**
        $L_{\min} = L_{potential}$;
        $P_{parent} = Tree.nodes(i)$;
      **end if**
    **end if**
  **end if**
**end for**

**Algorithm 6:** *Rewire.*

**Input:** $P_{new}, Tree$
**Output:** $Tree$
**for** each $Tree.nodes(i)$ **do**
  **if** $\|P_{new} - Tree.nodes(i)\| < R_{near}$ **then**
    $L_{potential} = Length(P_{new}) + \|P_{new} - Tree.nodes(i)\|$;
    **if** $path\_feasible(P_{new}, Tree.nodes(i))$ **then**
      **if** $L_{potential} < Length(Tree.nodes(i))$ **then**
        $Tree.nearest(i) = P_{new}$;
      **end if**
    **end if**
  **end if**
**end for**

Fig. 7(F), the updated path from the *Choose_Parent* and *Rewire* components show fewer redundancies.

While the optimized random tree using the *Choose_Parent* and *Rewire* components reduce local path redundancies, it may still produce winding paths overall. To tackle this, we referenced RRT*smart's beacon generation method and used the *Path_optimization* component to refine the existing feasible paths. The pseudocode for *Path_optimization* is in Algorithm 7, where $\varepsilon$ is a small value, and *get_nearest_index* returns the index of $Tree.nearest$ in $Tree.nodes$.

**Algorithm 7:** *Path_optimization.*

**Input:** $Tree, S_{init}, S_{goal},$
**Output:** $S$
$S = \varnothing$;
$S \leftarrow S_{goal}$;
$P_{beacon} = Tree.nodes(i)$, where $Tree.nodes(i)$ is closest to path point $S_{goal}$;
$P_{old\_beacon} = P_{beacon}$;
**while** $\|P_{beacon} - S_{init}\| \ge \varepsilon$ **then**
  **if** $path\_feasible(P_{beacon}, P_{old\_beacon})$ **then**
    **if** $\sim path\_feasible(Tree.nodes(get\_nearest\_index(P_{beacon})), P_{old\_beacon})$ **then**
      $S \leftarrow P_{beacon}$;
      $P_{old\_beacon} = P_{beacon}$;
    **end if**
  **end if**
  $P_{beacon} = Tree.nodes(get\_nearest\_index(P_{beacon}))$;
**end while**
$S \leftarrow S_{init}$;

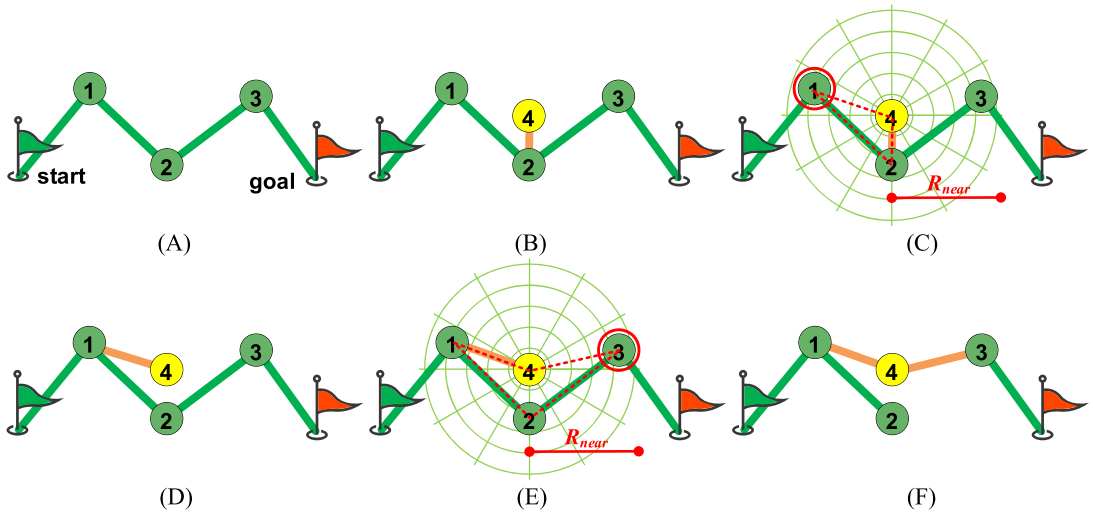



**Fig. 7.** Schematic diagram of path optimization strategy related components.

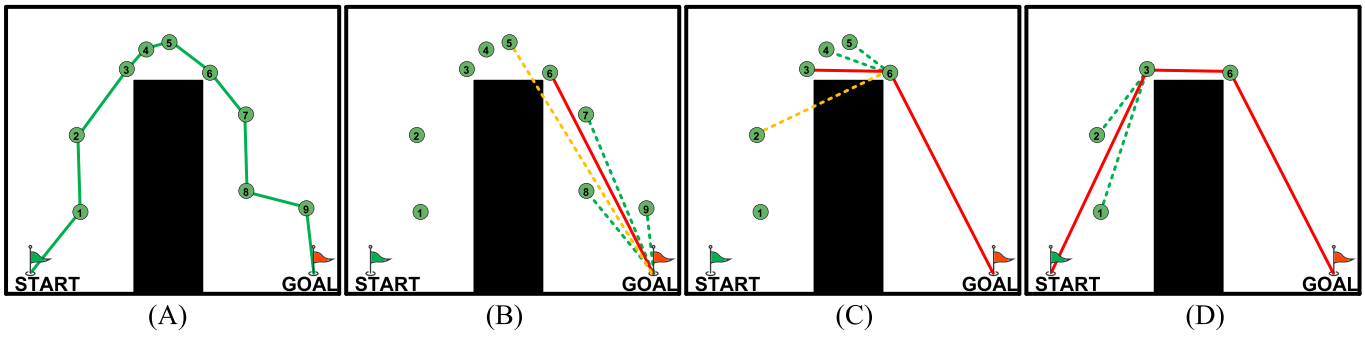



**Fig. 8.** Diagram of the *Path_optimization* component.

*Path_optimization* begins at the endpoint of existing feasible paths, checking the reachability of each node from its parent and adding eligible nodes to the final path set. As illustrated in Fig. 8(A), a path with nine nodes excludes the starting point and faces redundancy due to its winding nature. In Fig. 8(B), starting from the goal node, the *Path_optimization* component assesses connectivity to previous nodes, adding node 6 to set $S$ as it is reachable from nodes 6, 7, 8, and 9 but not from path node 5. Following this, in Fig. 8(B), it evaluates paths from node 6 and adds node 3 to set S after confirming reachable connections with nodes 3, 4, and 5, but not with node 2. The process concludes when node $S_{init}$ is checked, resulting in its addition to set $S$, as shown in Fig. 8(D).

### 3.5. HAURRT*smart algorithm

In Sections 3.1 to 3.4, we introduced the four key modified components of HAURRT*smart. This subsection presents the complete pseudocode for HAURRT*smart to clarify the interaction between different strategies. See Algorithm 8 for the pseudocode.

In terms of space complexity, the storage overhead of HAURRT*smart primarily stems from the storage of the tree structure *Tree*. Within this structure, $Tree.nodes$ represents the path node coordinate matrix with complexity $O(MaxIt)$; $Tree.nearest$ stores the parent node indices of each node with complexity $O(MaxIt)$, and the pseudorandom sequence $Hs$ follows a deterministic formula. Recording the index of each node in the $tree.nodes$ within $Hs$ result in a space complexity of $O(MaxIt)$ for $U$. All these structures exhibit linear storage characteristics. Therefore, the upper bound on the algorithm's space complexity is $O(MaxIt)$, resulting in linear space overhead. This constitutes the optimal space complexity for path planning algorithms, with no additional redundant storage.

To analyze the time complexity of HAURRT*smart, we need to consider its algorithmic flow and components. Inside the main loop of Algorithm 8: The functions such as $HAURRT^{*}smart\_get\_sampling\_node$, $HAURRT^{*}smart\_adaptive\_target\_bias$, and $path\_feasible$, which each have a time complexity of $O(1)$ as they do not involve parameterized loops. Although the $HAURRT^{*}smart\_extend\_nodes$ has a loop with parameter $Nn$, it has a theoretical upper bound related to map size, thus remaining $O(1)$. The $Choose\_Parent$ and $Rewire$functions require traversing all nodes, leading to a time complexity of $O(t)$, where $t$ is the current number of path nodes. As lines 8–13 of Algorithm 8 form a finite loop, the overall complexity remains $O(t)$. Outside this loop, $Path\_optimization$ has a complexity of $O(MaxIt)$ for traversing the nodes. Therefore, the overall time complexity of HAURRT*smart is $O(MaxIt^2)$.

## 4. Simulation results and analysis

This section compares the performance of HAURRT*smart through multiple experiments, each repeated 30 times due to simulation randomness. We compared it with three classic RRT-based algorithms and four improved RRT versions (see Table 2), following original parameter settings from the literature. Simulations were conducted on an i5-13600KF CPU @ 3.50GHz with 32GB RAM, using MATLAB 2023b.

In Table 2, the parameter settings for certain variables were not clearly stated in the original text. However, we optimized the algorithms using relevant methods to ensure a fair comparison while prioritizing adherence to the original literature's parameter settings. This paper selected 10 simulated environment maps, each $1000 \times 1000$ pixels in size, including simple environments, special complex

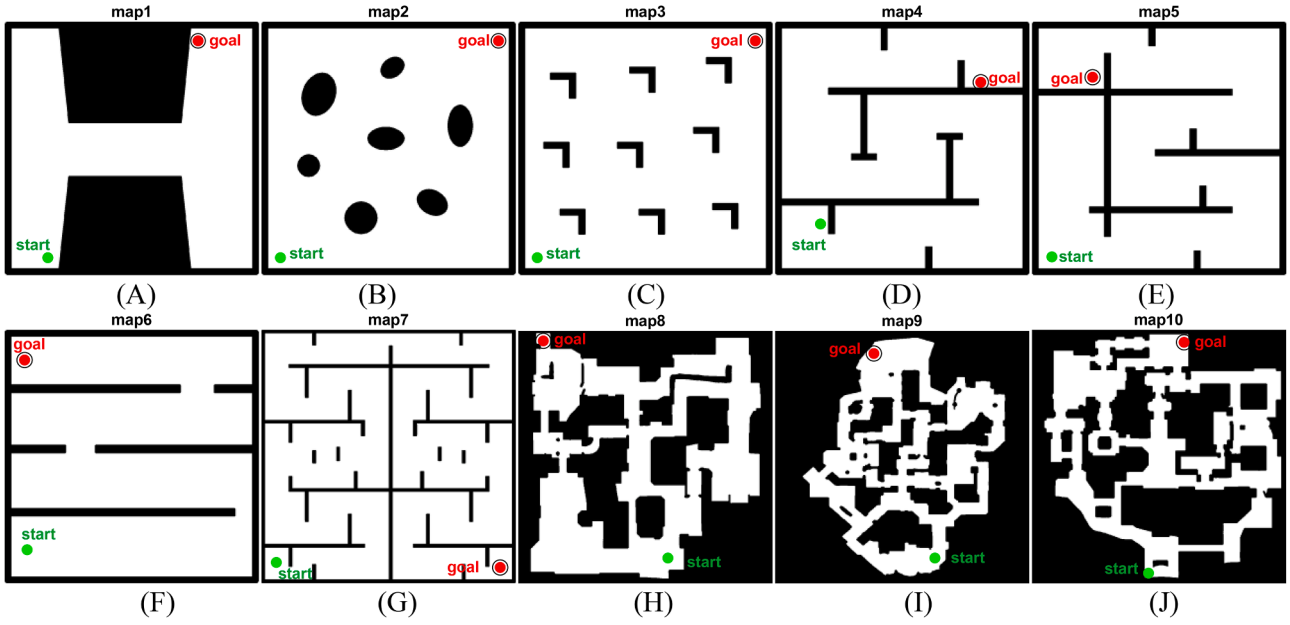



**Fig. 9.** All simulated maps.

**Algorithm 8:** HAURRT*smart.

**Input:** $M$(include $M_{free}$ and $M_{obs}$), $S_{init}, S_{goal}, R, MaxIt$
**Output:** $S = \{\sigma(t) | t = 1, 2, \ldots, Ns\}$
1: $Tree.nodes = \{S_{init}\}$, $Tree.nearest = \{Nan\}$; # $Nan$ is an element
2: **while** $t < MaxIt$ **do**
3: $P_{new}$, $P_{nearest} \leftarrow$ $HAURRT^*smart\_get\_sampling\_node(M, Hs, U, st, Tree)$;
4: $P_{rand}$, $P_{nearest} \leftarrow$ $HAURRT^*smart\_adaptive\_target\_bias(P_{new}, S_{init}, S_{goal})$;
5: $st = st + 1$;
6: **if** $path\_feasible(P_{new}, P_{nearest})$ **then**
7: $st = 0$; $Shoots \leftarrow$ $HAURRT^*smart\_extend\_nodes(Tree, P_{new}, P_{nearest})$;
8: **for** each $Shoots.nodes(i)$ **do**
9: $Shoots.nearest(i) =$ $Choose\_Parent(Shoots.nodes(i), Shoots.nearest(i), Tree)$;
10: $Tree.nodes \leftarrow \{Tree.nodes, Shoots.nodes(i)\}$;
11: $Tree.nearest \leftarrow \{Tree.nearest, Shoots.nearest(i)\}$;
12: $Tree = Rewire(Shoots.nodes(i), Tree)$;
13: **end for**
14: **end if**
15: **if** $\left\| P_{new} - S_{goal} \right\| \leq R$ **then**
16: $Tree.nodes \leftarrow \{Tree.nodes, S_{goal}\}$;
17: $Tree.nearest \leftarrow \{Tree.nearest, P_{new}\}$;
18: **break**;
19: **end if**
20: $t = t + 1$;
21: **end while**
22: $S \leftarrow Path\_optimization(Tree, S_{init}, S_{goal})$;

passageways, extremely complex maze, and environments simulating reality. The characteristics and reasons for their use are summarized in Table 3. We establish the start point and goal point for 10 simulated maps, as shown in Fig. 9.

### 4.1. Extremely complex maze

The starting point coordinates and maximum iteration counts for the 10 simulated maps are shown in Table 4, using a pixel coordinate system. The algorithm's single-step expansion distance is set to 5% of the map size. The acceptance radius for the goal point is 30 pixels, and a feasible path is found when a node in the random tree enters this radius.

The simulation experiment analyzed the performance of HAURRT*smart regarding path search efficiency and several performance metrics. Key evaluation metrics include $N_{init}$ (iterations to find the first feasible solution), $T_{init}$ (runtime for the first feasible solution), and $coverage\ efficiency$ (ratio of coverage rate to number of nodes). $N_{init}$ serves as an efficiency metric for the algorithm; a lower value indicates fewer path sampling points. This makes the metric particularly important for agents operating on low-configuration embedded systems. Similarly, $T_{init}$ serves as an efficiency metric that directly reflects the minimum time the algorithm takes to identify a viable path under identical conditions. This metric is especially critical when deploying the algorithm in scenarios that demand quick responses, such as rescue operations or military engagements. The $coverage\ efficiency$ serves as an indicator of the algorithm's redundancy. In highly complex, high-dimensional situations, this metric deserves careful consideration. Excessive reliance on sampling for path planning in such environments can lead to significant performance waste in the agent's use of the path solver.

We also examined the distribution of sampling points and the quality of optimal paths across various maps, comparing the performance differences between the algorithms. During the growth process of random trees, a balanced and uniform sampling strategy crucial for algorithm performance. We estimated the coverage rate of random trees on each map by applying grid processing. Coverage is the ratio of pixel blocks with sampling points in the reachable area to the total number of reachable pixel blocks. Since this value is a rough estimate based on pixel block counts, we selected the resolution using the criterion that obstacles should be clearly visible.

**Table 2**
Comparison of Different RRT*-based Algorithms.

| Reference | Algorithm Selection Criteria | Parameters |
|---|---|---|
| AE-RRT*[63] | Newly published, applicable to high-dimensional spaces, dynamic environments, and complex structures, with lower complexity. | $p_{\text{target}} = 0.1$ |
| CDRT-RRT*[81] | Recently published, low collision computation, delay re-routing strategy reduces branch redundancy, provides temporary paths under unreachable conditions, but requires pre-processing of the environment. | $p_{\text{th}} = 0.3$; $r_{\text{th}} = 15$ |
| Cloud RRT*[80] | Initializing the workspace based on a generalized Voronoi diagram allows global sampling to be maintained while continuously improving the optimal solution. | $\alpha = 0.7$ |
| Informed RRT*[60] | Classical improved RRT algorithm. | ~ |
| RRT*[101] | Classical improved RRT algorithm. | bais = 0.1 |
| RRT*smart[62] | Classical improved RRT algorithm. | $\text{beacon}_{\text{ratio}} = 0.33$; bais = 0.1 |
| TQRRT*[86] | Newly published, high convergence speed, combining the advantages of multiple algorithms, capable of removing redundant parent nodes. | $tcr_{\text{limit}} = 0.7$; $\text{depth}_{\text{limit}} = 3$; $\beta_q = 0.6$; $\gamma_1 = 0.3$; $\gamma_2 = 0.6$; expansion = 0.3; bais = 0.1 |
| HAURRT*smart | Proposed in this paper. | ~ |

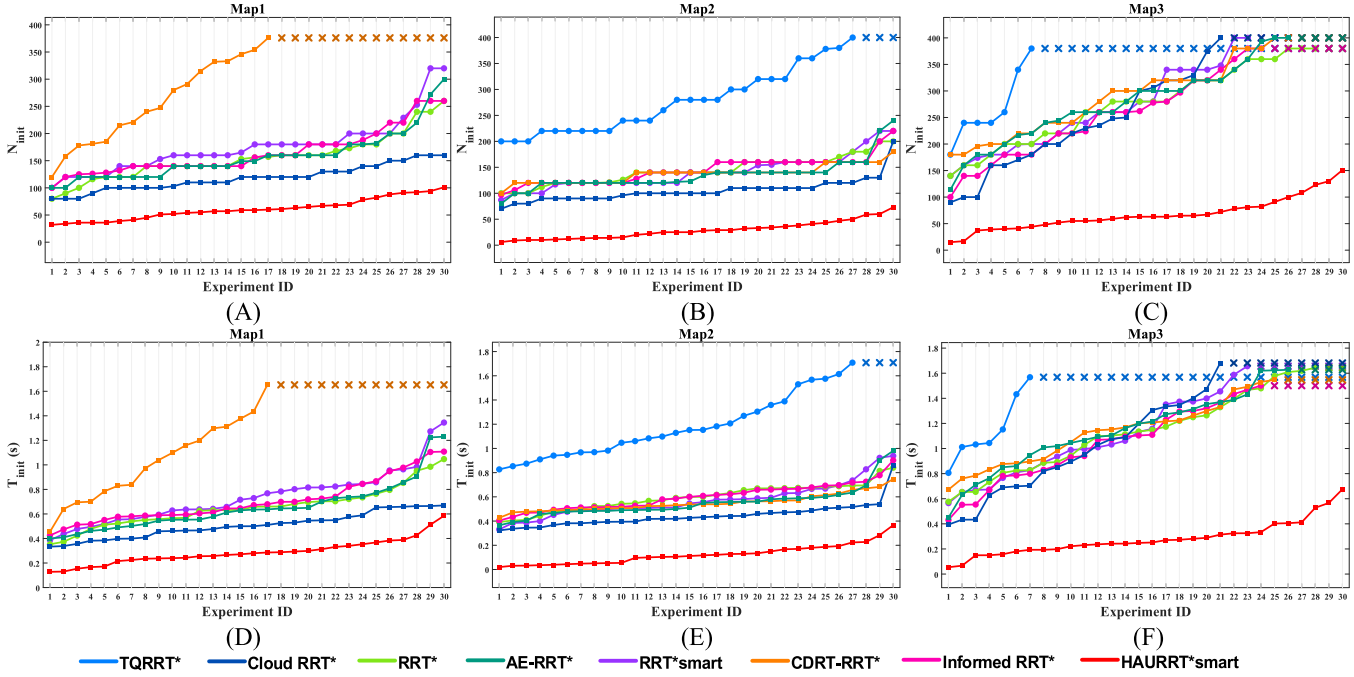



**Fig. 10.** $N_{init}$ and $T_{init}$ in 3 simple environments.

### 4.2. Simple environment

This paper examines three simple environments with basic obstacle structures, chosen to test the algorithm's pathfinding efficiency. Map1 features two enormous obstacles, which hinder random tree growth by generating sampling points on them. Map2 consists of several smooth obstacles, while Map3 includes multiple right-angled obstacles that can trap the algorithm during transitions. The algorithm is limited to 400 iterations to evaluate its rapid exploration capabilities in these settings.

Fig. 10 displays $N_{init}$ and $T_{init}$ across three Simple environments with 30 repeated experiments. Experiments that failed to produce an initial solution are marked with '×', and TQRRT* is excluded due to infeasibility in Map1. Table 5 and Table 6 display the statistical results for $N_{init}$ and $T_{init}$ across repeated experiments. Bold text highlights the minimum value of the Best, the Worst, and the Mean result, the minimum value of the standard deviation, as well as values that demonstrate significant differences in the Wilcoxon rank-sum test results at a 5% significance level. In Fig. 10, HAURRT*smart achieves $T_{init}$ at only 57.9%, 27.8%, and 24.7% of the suboptimal algorithm across the maps. Note that the preprocessing time is not included in $T_{init}$, suggesting longer times for Cloud RRT* and CDRT-RRT*. In Map3, other algorithms struggled to find feasible solutions in some experiments, highlighting the robustness of HAURRT*smart in various environments. From Table 5 and Table 6, in simple environments, HAURRT*smart outperforms comparison algorithms in optimal, worst-case, and average results, showing statistically significant differences. While $T_{init}$ ranked second in Map2 and had suboptimal stability, HAURRT*smart maintained optimal stability across all other scenarios.

Fig. 11 presents the distribution of sampling points for eight algorithms in three simulated environments, displaying only the results

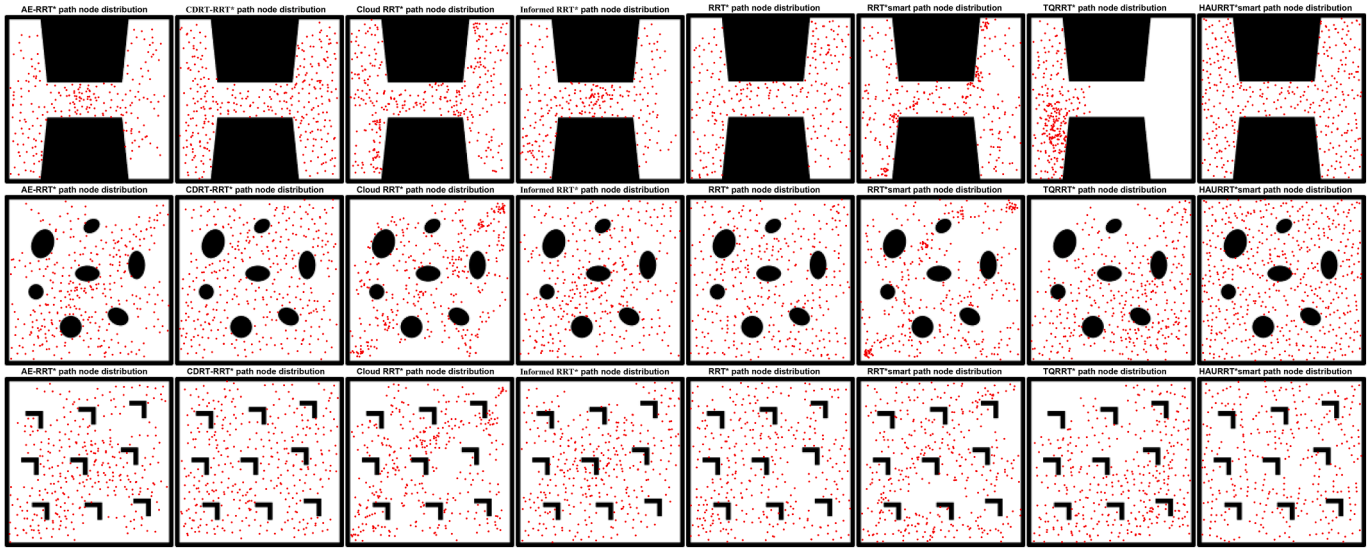



**Fig. 11.** Path node distribution on three simple maps.

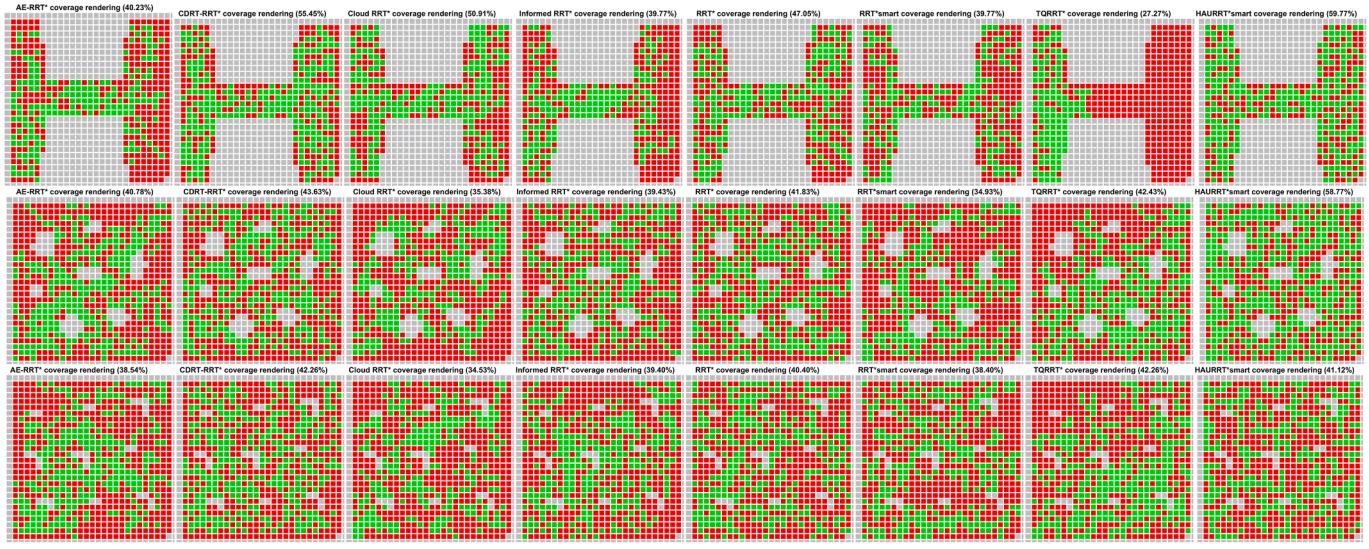



**Fig. 12.** Path node coverage rendering maps on three simple maps.

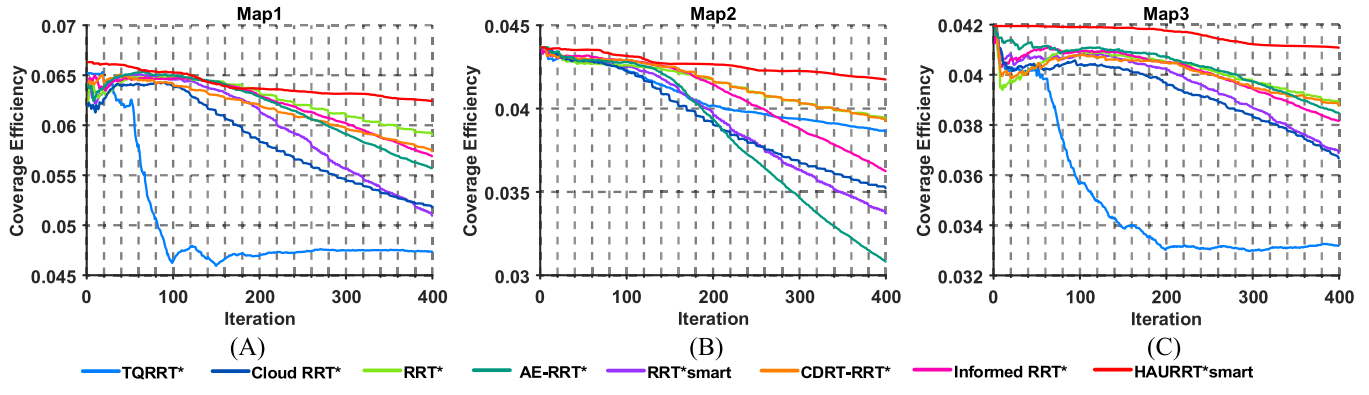



**Fig. 13.** Coverage efficiency trends on three simple maps.

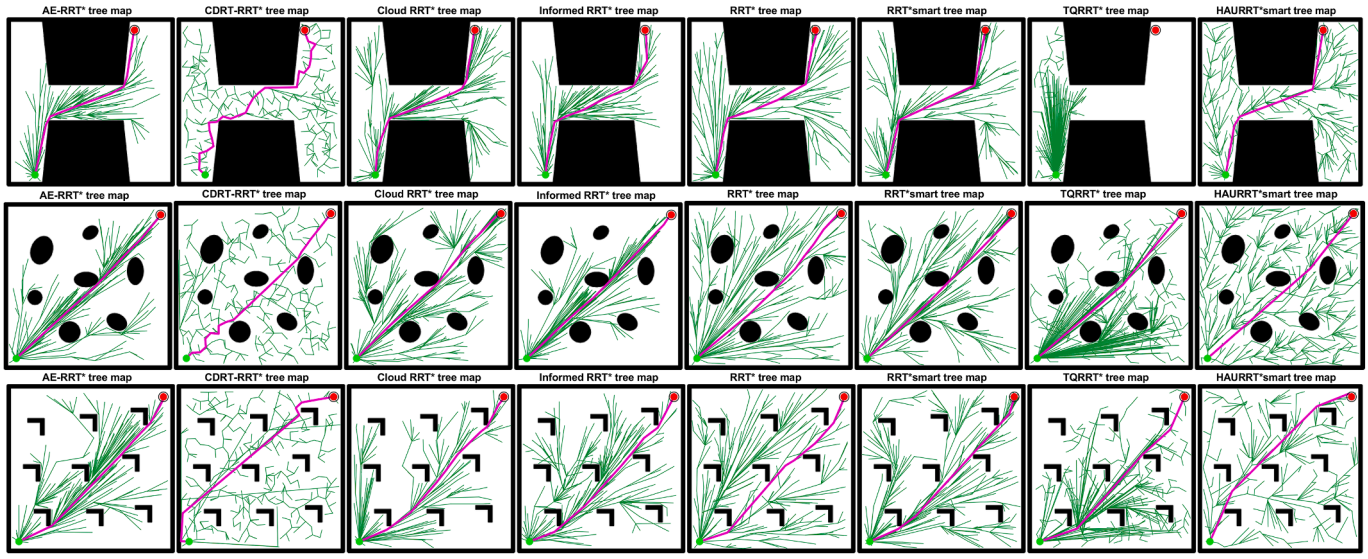



**Fig. 14.** Optimal planning results on three simple maps.

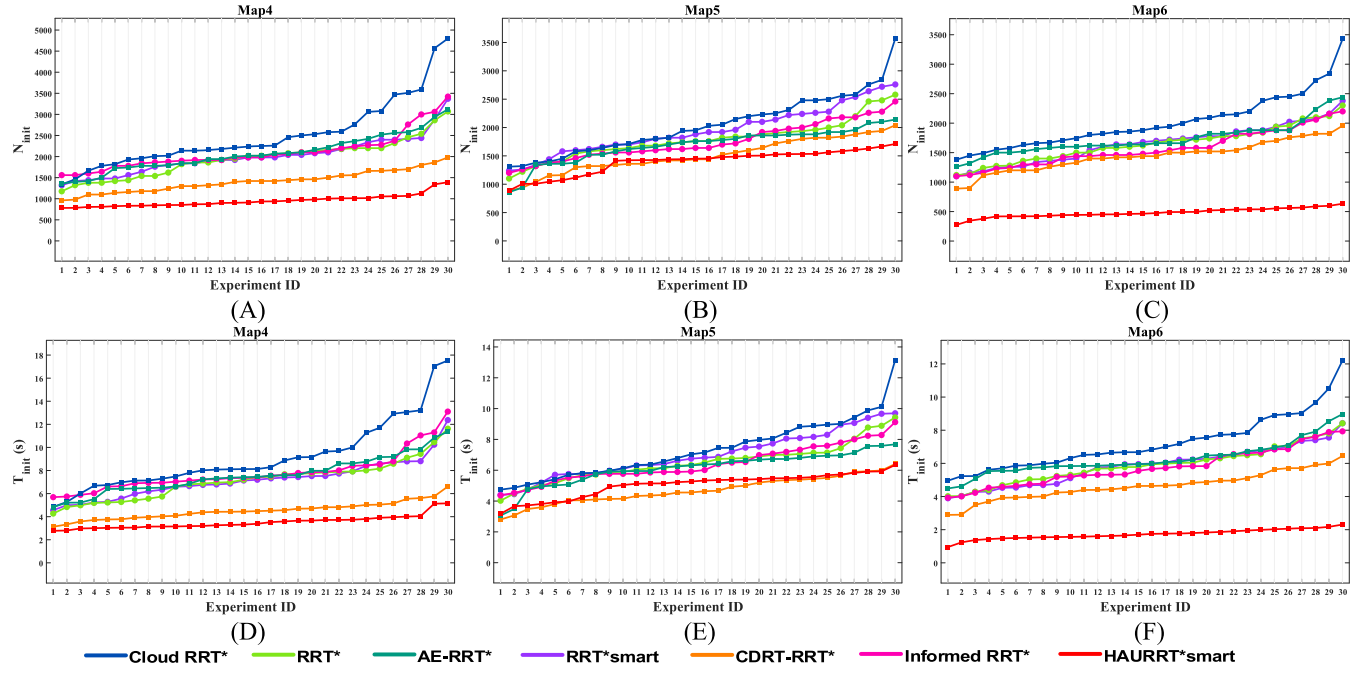



**Fig. 15.** $N_{init}$ and $T_{init}$ in 3 special complex passageways.

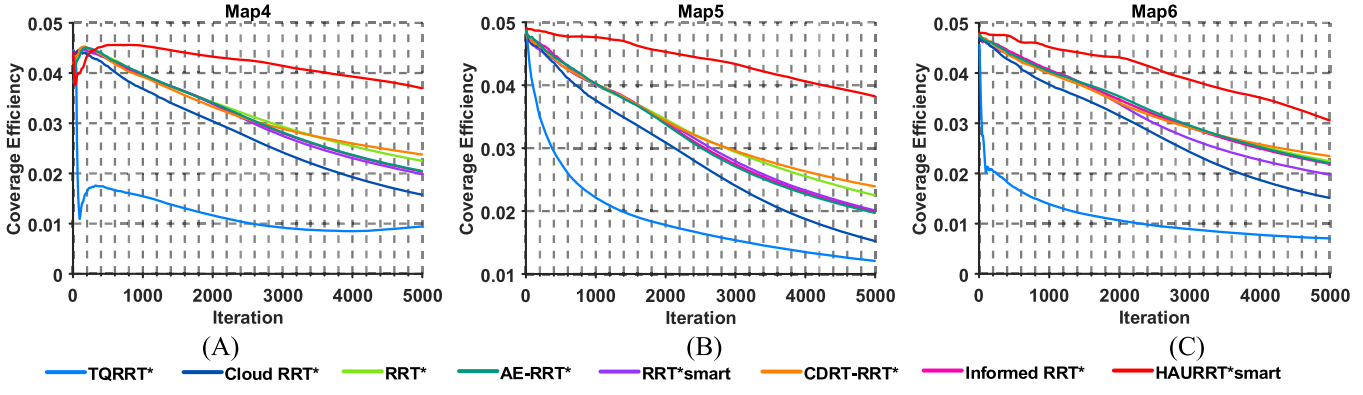



**Fig. 16.** Coverage efficiency trends on 3 special complex passageways.

**Table 3**
Features of simulated maps and reasons for selection.

| Type | Map ID | Obstacle features | Criteria for environmental selection |
|---|---|---|---|
| Simple environment | 1 | Extremely large obstacles | Test the ability to converge quickly and its ability to balance environmental exploration and path optimization. |
| | 2 | Dispersed, smoothly obstacles | |
| | 3 | Dispersed, irregularly obstacles | |
| Special complex passageway | 4 | Spiral | Test the design completeness for various environment structures, and test the existence of imbalances in environment exploration. |
| | 5 | Impassable route exists | |
| | 6 | Back and forth, long distance | |
| Extremely complex maze | 7 | Hybrid, large-scale | Testing the adaptability of algorithms to extremely complex environments. |
| Environment simulating reality | 8 | Hybrid, large-scale, realistic simulation, multiple feasible paths exist | Test the adaptability of the algorithm to the actual environment. These maps are from the game CS2. |
| | 9 | | |
| | 10 | | |

**Table 4**
Relevant Information about Parameters Corresponding to Maps.

| Map ID | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| Start | [180, 980] | [80, 980] | [80, 980] | [180, 800] | [80, 930] | [90, 870] | [60, 920] | [590, 900] | [630, 900] | [460, 960] |
| Goal | [800, 80] | [980, 80] | [980, 80] | [810, 240] | [240, 220] | [80, 120] | [940, 940] | [100, 40] | [390, 90] | [600, 50] |
| Iteration | 400 | 400 | 400 | 5000 | 5000 | 5000 | 80,000 | 5000 | 5000 | 5000 |

**Table 5**
Statistical Results for Indicator $N_{init}$ in the Simple Environment.

| | Map | AE-RRT* | CDRT-RRT* | CloudRRT* | InformedRRT* | RRT* | RRT*smart | TQRRT* | HAURRT*smart |
|---|---|---|---|---|---|---|---|---|---|
| Best | 1 | ~ | 100 | 80 | 100 | 80 | 120 | 100 | **30** |
| | 2 | 200 | 90 | 100 | 100 | 70 | 100 | 80 | **10** |
| | 3 | 180 | 140 | 140 | 100 | 90 | 180 | 110 | **20** |
| Worst | 1 | ~ | 320 | 260 | 260 | 160 | 380 | 300 | **100** |
| | 2 | 400 | 220 | 200 | 220 | 200 | 180 | 240 | **70** |
| | 3 | 380 | 400 | 380 | 380 | 400 | 400 | 400 | **150** |
| Mean | 1 | ~ | 176.67 | 156.00 | 165.67 | 118.00 | 311.00 | 157.67 | **61.33** |
| | 2 | 288.67 | 140.67 | 143.67 | 144.67 | 105.33 | 137.33 | 135.33 | **29.00** |
| | 3 | 354.00 | 292.67 | 280.00 | 274.00 | 283.67 | 300.67 | 290.67 | **68.00** |
| Std | 1 | ~ | 5.13E+01 | 4.18E+01 | 4.26E+01 | 2.35E+01 | 8.42E+01 | 4.55E+01 | **1.91E+01** |
| | 2 | 6.94E+01 | 3.31E+01 | 2.72E+01 | 2.62E+01 | 2.29E+01 | 1.72E+01 | 3.14E+01 | **1.71E+01** |
| | 3 | 5.71E+01 | 9.05E+01 | 7.59E+01 | 8.85E+01 | 1.07E+02 | 7.57E+01 | 8.16E+01 | **3.03E+01** |
| p | 1 | ~ | **2.73E-11** | **6.65E-11** | **2.64E-11** | **2.70E-10** | **2.07E-11** | **2.84E-11** | ~ |
| | 2 | **2.46E-11** | **2.22E-11** | **2.31E-11** | **2.00E-11** | **2.41E-11** | **1.75E-11** | **1.92E-11** | ~ |
| | 3 | **5.93E-12** | **2.83E-11** | **2.99E-11** | **5.03E-11** | **1.06E-10** | **2.65E-11** | **3.83E-11** | ~ |

**Table 6**
Statistical Results for Indicator $T_{init}$ in the Simple Environment.

| | Map | AE-RRT* | CDRT-RRT* | CloudRRT* | InformedRRT* | RRT* | RRT*smart | TQRRT* | HAURRT*smart |
|---|---|---|---|---|---|---|---|---|---|
| Best | 1 | ~ | 0.39 | 0.35 | 0.42 | 0.33 | 0.46 | 0.40 | **0.13** |
| | 2 | 0.83 | 0.33 | 0.39 | 0.40 | 0.32 | 0.43 | 0.36 | **0.02** |
| | 3 | 0.81 | 0.57 | 0.58 | 0.43 | 0.39 | 0.67 | 0.45 | **0.05** |
| Worst | 1 | ~ | 1.34 | 1.05 | 1.11 | 3.66 | 1.65 | 1.23 | **0.59** |
| | 2 | 1.71 | 0.94 | 0.84 | 0.90 | 0.86 | 0.75 | 0.98 | **0.37** |
| | 3 | 1.57 | 1.66 | 1.64 | 1.50 | 1.68 | 1.55 | 1.63 | **0.68** |
| Mean | 1 | ~ | 0.74 | 0.65 | 0.70 | 1.11 | 1.30 | 0.65 | **0.29** |
| | 2 | 1.23 | 0.57 | 0.60 | 0.60 | 0.44 | 0.55 | 0.55 | **0.12** |
| | 3 | 1.47 | 1.20 | 1.16 | 1.12 | 1.19 | 1.19 | 1.20 | **0.28** |
| Std | 1 | ~ | 2.22E-01 | 1.65E-01 | 1.82E-01 | 1.23E+00 | 3.98E-01 | 2.02E-01 | **1.04E-01** |
| | 2 | 2.91E-01 | 1.45E-01 | 1.12E-01 | 1.11E-01 | 9.83E-02 | **7.16E-02** | 1.29E-01 | 8.17E-02 |
| | 3 | 2.16E-01 | 3.75E-01 | 3.32E-01 | 3.36E-01 | 4.50E-01 | 2.79E-01 | 3.31E-01 | **1.37E-01** |
| p | 1 | ~ | **1.21E-10** | **3.16E-10** | **8.15E-11** | **3.20E-09** | **2.77E-11** | **2.61E-10** | ~ |
| | 2 | **3.00E-11** | **3.34E-11** | **3.02E-11** | **3.02E-11** | **4.50E-11** | **3.02E-11** | **3.34E-11** | ~ |
| | 3 | **6.48E-12** | **4.72E-11** | **4.07E-11** | **6.47E-11** | **9.91E-11** | **3.27E-11** | **4.45E-11** | ~ |

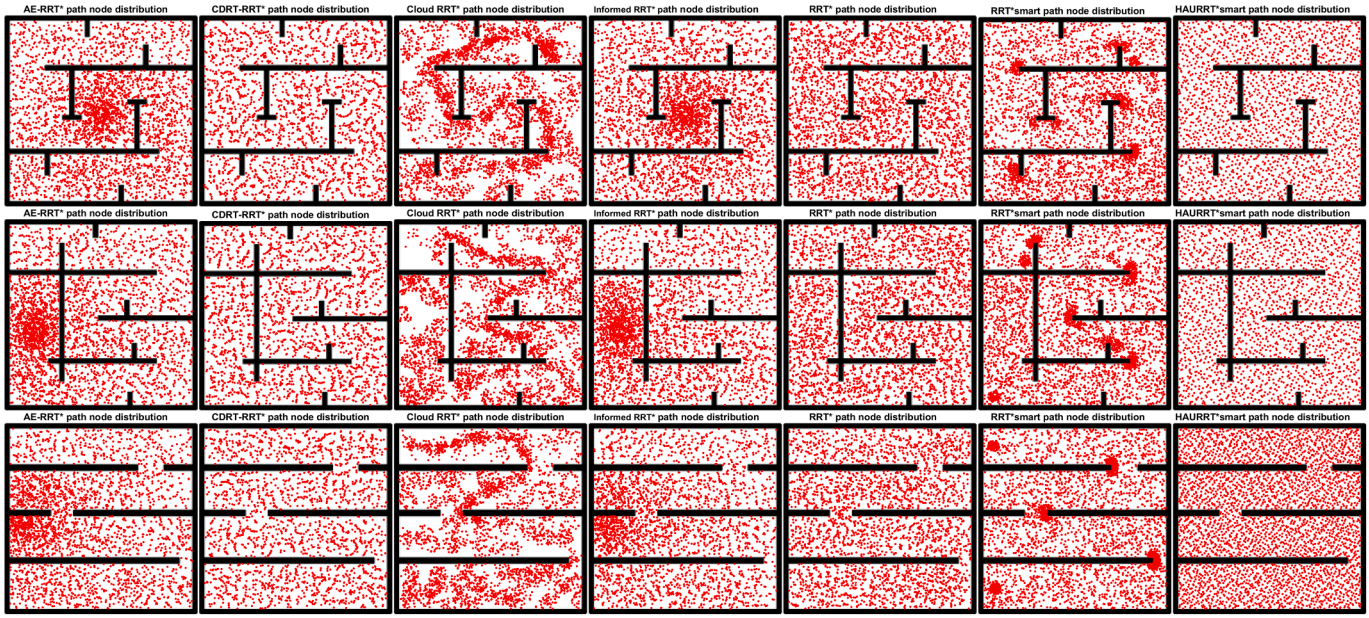



**Fig. 17.** Path node distribution on 3 special complex passageways.


**Fig. 18.** Path node coverage rendering maps on 3 special complex passageways.

with the shortest generation paths. We discretized those regions to create coverage rendering maps (Fig. 12), where grey indicates unreachable areas, green shows reachable regions with sampling points, and red highlights reachable areas without points. Notably, in Fig. 11, AE-RRT*, CDRT-RRT*, Cloud RRT*, and Informed RRT* show a lack of sampling at edges, while AE-RRT*, Cloud RRT*, Informed RRT*, and TQRRT* miss sampling at corners. Additionally, AE-RRT*, Cloud RRT*, RRT*, and RRT*smart have internal areas with missing clusters. In contrast, HAURRT*smart demonstrates a uniform sampling distribution with fewer redundancies, achieving coverage rates of 59.77% and 58.77% on Map1 and Map2, and 41.12% on Map3. The Halton sequence controls sampling distribution in HAURRT*smart, enhancing uniformity and filling structural gaps effectively, as further validated by the results in Fig. 12.

In Fig. 10, HAURRT*smart has a smaller $N_{init}$ than the comparison algorithm, indicating it finds an initial feasible solution quickly. While HAURRT*smart may generate multiple sampling points in one iteration, the results for $T_{init}$ have already demonstrated that its performance advantage is not solely reliant on this. To ensure the rigor of the experimental results, we investigated the trend of coverage efficiency across iterations for the eight algorithms, with the results shown in Fig. 13. Fig. 13 shows that HAURRT*smart maintains high coverage efficiency across all iterations, effectively distributing sampling points throughout the reachable region with minimal path node redundancy.

RRT and its derivatives prioritize adaptability to unknown environments over path length minimization. Fig. 14 presents the optimal planning results from 30 repeated experiments. All algorithms, except

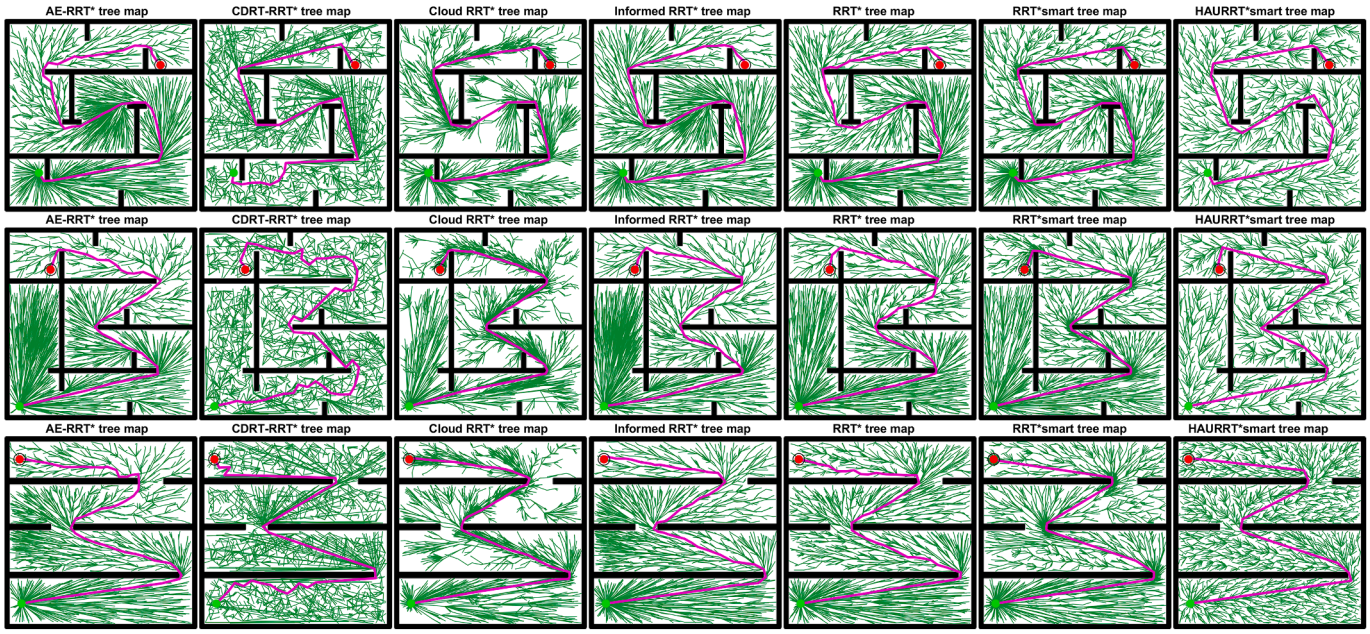



**Fig. 19.** Optimal planning results on 3 special complex passageways.

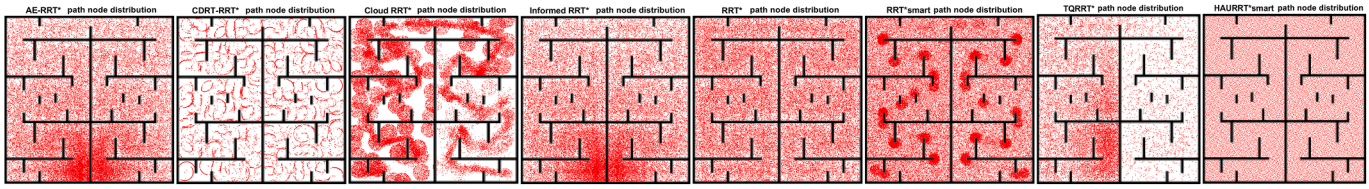



**Fig. 20.** Path node distribution on extremely complex maze.

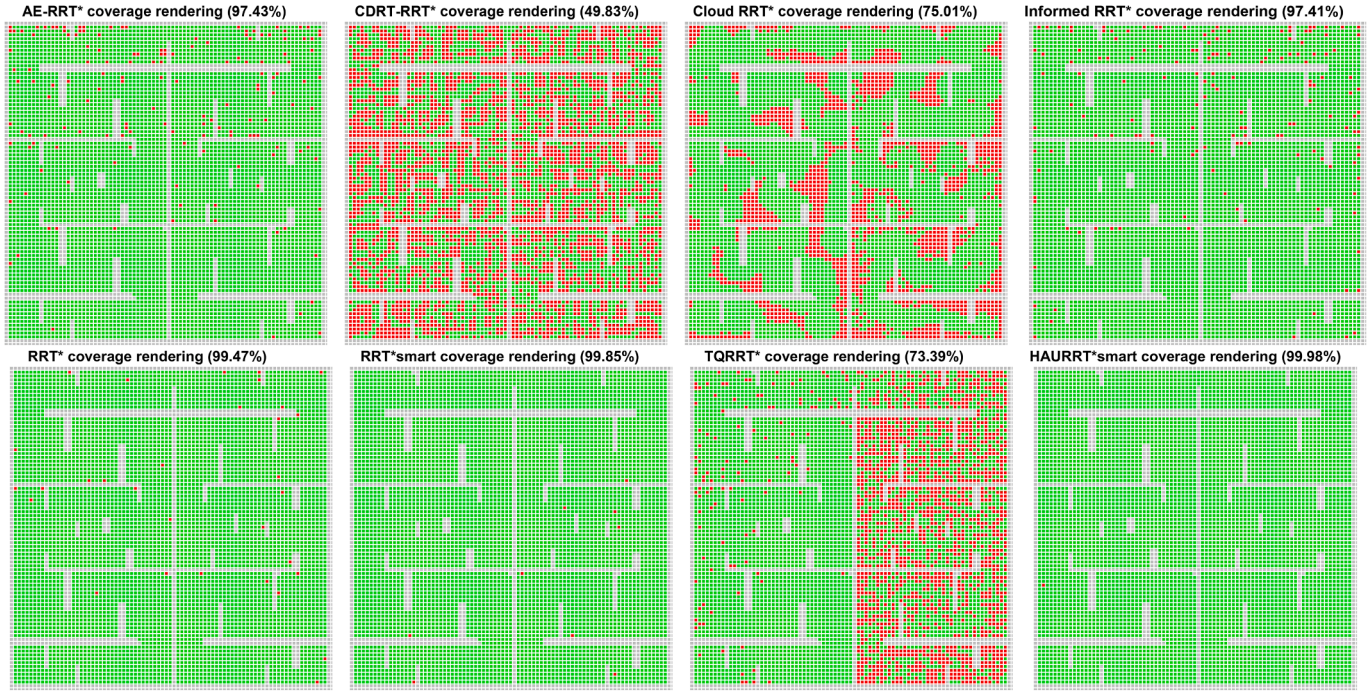



**Fig. 21.** Path node coverage rendering maps on extremely complex maze.

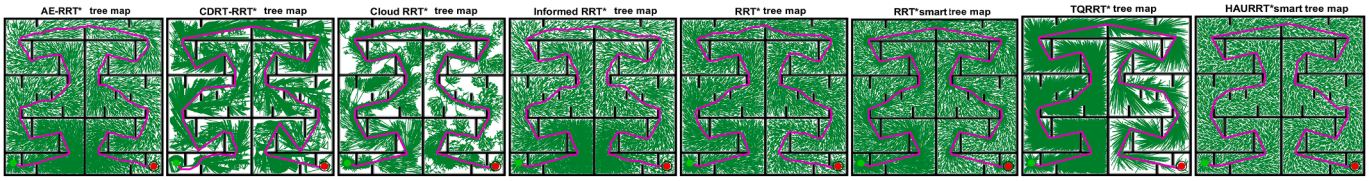



**Fig. 22.** Optimal planning results on extremely complex maze.

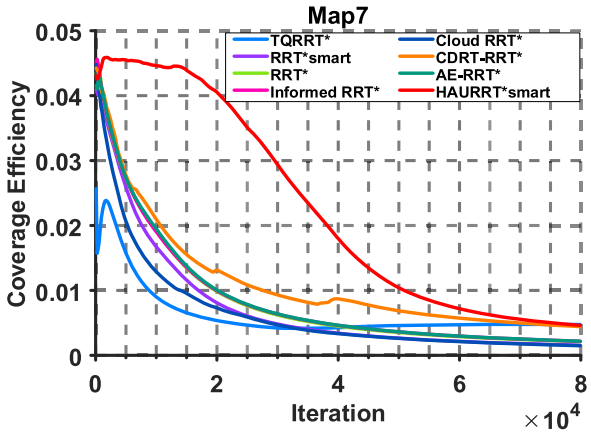



**Fig. 23.** Coverage efficiency trends on extremely complex maze.

CDRT-RRT*, show minor differences. TQRRT* struggles to balance exploration and path optimization, resulting in too many sampling points near the starting point, thus failing to generate feasible paths in Map1. AE-RRT* and TQRRT* share similar target bias strategies, leading to concentrated random trees near the starting connection.

### 4.3. Special complex passageway

This paper examines three Special complex passageway structures that present challenges in pathfinding. Map4 features an 'S'-shaped spiral pipe, where the algorithm must fill the entire pipe with a random tree, navigating through sequences of approaching and moving away from the endpoint. Map5 features a deceptive pipe terrain where excessive growth toward the target can trap the tree in a lower-left cavity. Map6 consists of four large cavities linked by small gaps, requiring careful traversal through these gaps to reach the target. Due to the increased complexity of these simulated environments, the maximum iteration count is set to 5000 to assess the algorithm's overall performance in handling these complex structures.

Fig. 15 displays the $N_{init}$ and $T_{init}$ values for three Special complex passageways, with the x-axis indicating 30 repeated experiments. TQRRT* is excluded due to its inability to generate feasible solutions in this environment. In Fig. 15, HAURRT*smart significantly decreases iterations and iteration time while effectively finding paths. On Map4 and Map6, HAURRT*smart's $T_{init}$ is only 77.8% and 37.1% of CDRT-RRT*'s $T_{init}$, respectively. Although Map5 shows a 7.4% increase in HAURRT*smart's $T_{init}$ compared to CDRT-RRT*, the latter still experiences additional initialization time. Overall, HAURRT*smart consistently maintains an efficiency advantage despite varying environmental complexities.

Table 7 and Table 8 display the statistical results for $N_{init}$ and $T_{init}$ across repeated experiments. Bold text highlights the minimum value of the Best, the Worst, and the Mean result, the minimum value of the standard deviation, as well as values that demonstrate significant differences in the Wilcoxon rank-sum test results at a 5% significance level. Statistical results show that on Map5, HAURRT*smart and RRT*smart perform similarly on $N_{init}$ and $T_{init}$ metrics. However, on other maps, HAURRT*smart significantly outperforms the comparison algorithms across all indicators.

In the Special complex passageway environment, the coverage efficiency of the 8 algorithms varies with iterations, as shown in Fig. 16. Despite a larger number of iterations, HAURRT*smart consistently outperforms the others in coverage efficiency, and this advantage showing little change with iteration or environmental complexity. HAURRT*smart significantly enhances the distribution of sampling points, reducing redundancy and ensuring a balanced, uniform distribution.

Fig. 17 displays the distribution of sampling points for seven algorithms in three complex passageway environments, with coverage maps in Fig. 18. In Fig. 17, AE-RRT*, Cloud RRT*, Informed RRT*, and RRT*smart show dense sampling regions, particularly Informed RRT* near optimal path inflexion points due to its sampling strategy. AE-RRT* and Informed RRT* excessively sample areas away from endpoints in Map4 and the deceptive structure of Map5, indicating limitations in handling complex environments. Cloud RRT* increases sampling frequency near the optimal path, improving accuracy but restricting exploration, leading to under sampled areas. In Fig. 18, HAURRT*smart achieves 100% coverage much faster than other algorithms, especially in Map4 and Map6, completing coverage within 5000 iterations. This result highlights that only HAURRT*smart effectively demonstrates probabilistic completeness in simulations, while other algorithms need more iterations. HAURRT*smart avoids under-sampling and redundancy and demonstrates an efficient strategy, unlike the other algorithms.

Fig. 19 displays the optimal planning results of seven algorithms across 30 experiments. In Fig. 19, Paths from AE-RRT*, CDRT-RRT*, Informed RRT*, and RRT* show winding sections, with AE-RRT*, CDRT-RRT*, and RRT* demonstrating poor path quality near the starting point. In contrast, Cloud RRT*, RRT*smart, and HAURRT*smart deliver high-quality, balanced paths. HAURRT*smart maintains high search efficiency in complex environments and continues to generate quality paths.

The performance of path search algorithms should remain stable regardless of terrain size and complexity. This paper tests scalability using a large maze scene. In Map7, the starting points are located at the lower left and lower right corners of the map.

Fig. 20 shows the distribution of sampling points for seven algorithms in a complex maze, with their coverage maps in Fig. 21. Fig. 22 presents optimal planning results from 30 experiments. In this environment, AE-RRT* and Informed RRT* leave many small unexplored areas far from the start and lack coverage near obstacle edges. CDRT-RRT*, Cloud RRT*, and TQRRT* miss large pixel regions, while TQRRT* shows imbalances near both the start and end points. CDRT-RRT* and Cloud RRT* have significant gaps in their sampling patterns. AE-RRT*, Informed RRT*, and TQRRT* face significant search imbalance issues. AE-RRT*, Informed RRT*, and RRT* show path curvature, but there is little difference in path length among them. In comparison, HAURRT*smart maintains totally balanced sampling in complex scenarios.

In a complex maze environment, the coverage efficiency of eight algorithms varies with iterations, as shown in Fig. 23. CDRT-RRT*, Cloud RRT*, and TQRRT* had maximum coverage rates below 80% and are not analyzed further. The remaining algorithms achieved 95% or more coverage. HAURRT*smart maintained higher node coverage efficiency with fewer redundant nodes throughout the whole process. The results reflect long-term performance changes.

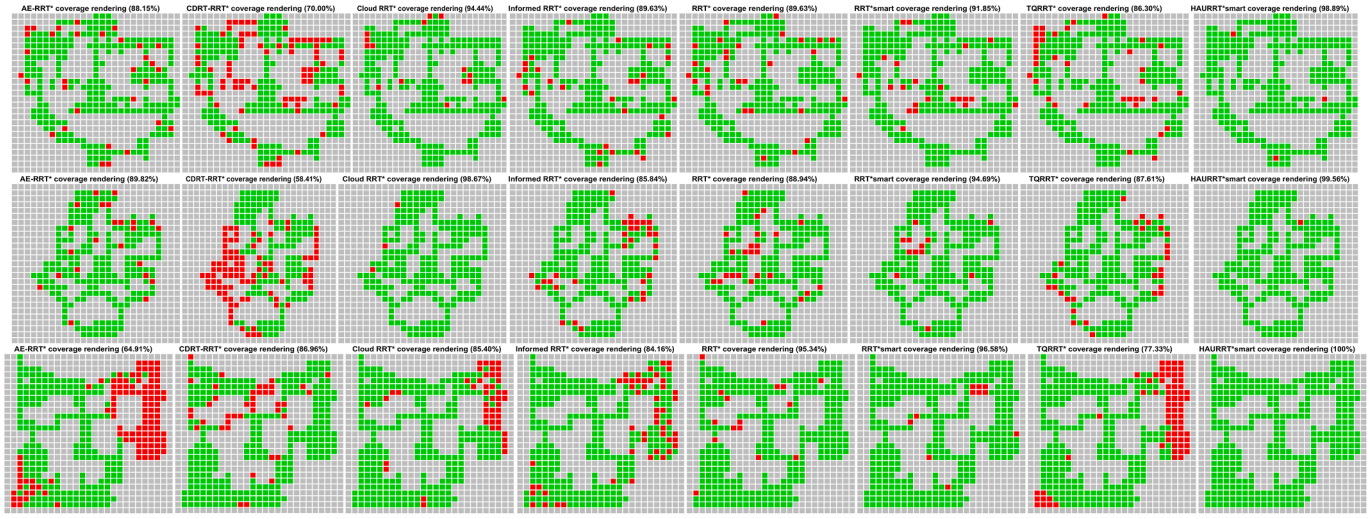



**Fig. 24.** Path node coverage rendering maps on 3 environments simulating reality.

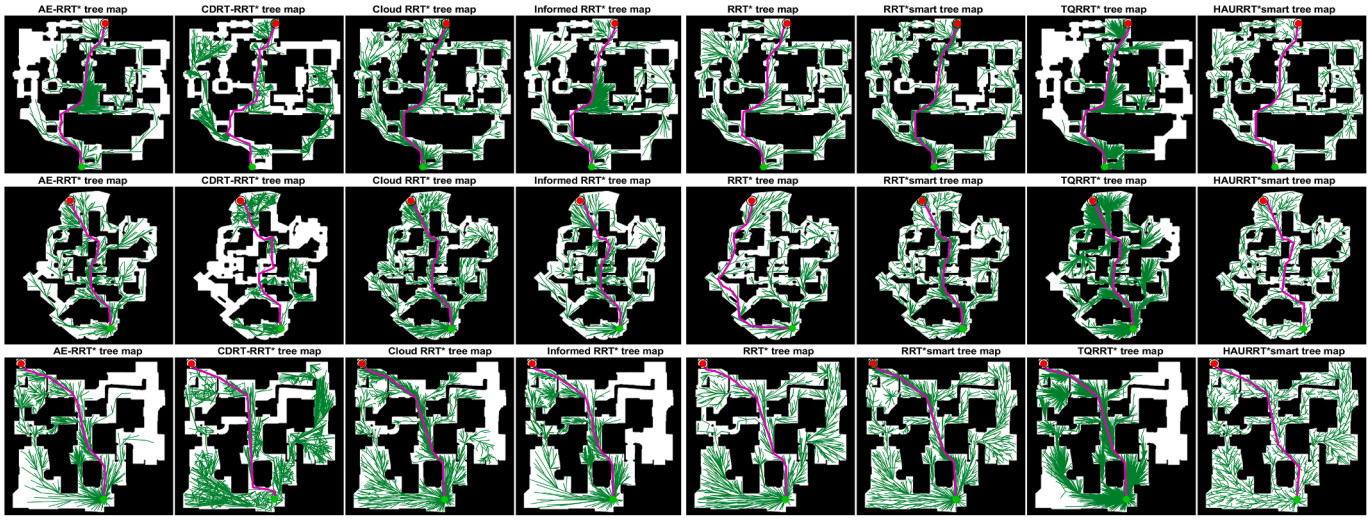



**Fig. 25.** Optimal planning results on 3 environments simulating reality.

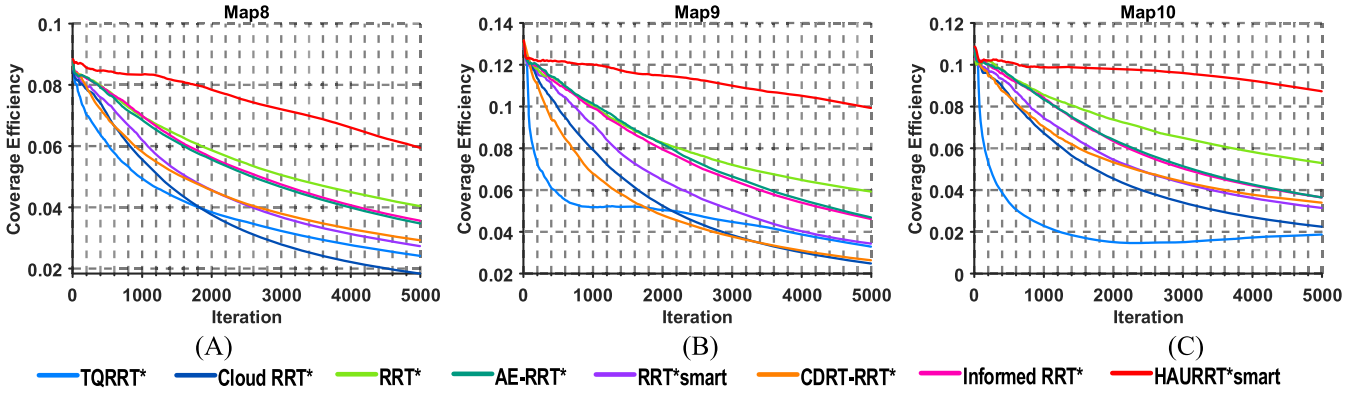



**Fig. 26.** Coverage efficiency trends on 3 environments simulating reality.

### *4.4. Environment simulating reality*

Algorithm design should focus on practical applications. This paper introduces three simulated scenarios that reflect real-world environments, featuring enhanced connectivity and a variety of path choices. The paths are irregular, including both narrow, challenging areas and easily expandable spaces.

Fig. 24 displays the coverage maps of eight algorithms in a reality-simulating environment. Fig. 25 presents the optimal planning result. Results reveal that AE-RRT*, CDRT-RRT*, Cloud RRT*, Informed RRT*,

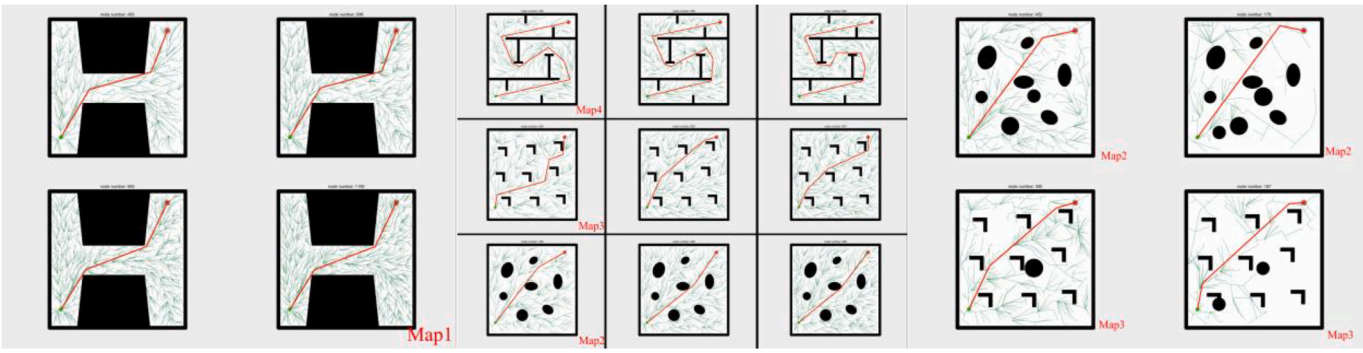



**Fig. 27.** Screenshots of support material in dynamic environment.

**Table 7**
Statistical Results for Indicator $N_{init}$ in the Special complex passageway.

| | Map | CDRT-RRT* | CloudRRT* | InformedRRT* | RRT* | RRT*smart | TQRRT* | HAURRT*smart |
|---|---|---|---|---|---|---|---|---|
| Best | 4 | 1320 | 1180 | 1560 | 1340 | 960 | 1360 | **780** |
| | 5 | 1240 | 1100 | 1200 | 1310 | 880 | **860** | 890 |
| | 6 | 1100 | 1120 | 1100 | 1380 | 890 | 1260 | **270** |
| Worst | 4 | 3370 | 3070 | 3420 | 4800 | 1980 | 3120 | **1390** |
| | 5 | 2760 | 2580 | 2460 | 3570 | 2040 | 2140 | **1710** |
| | 6 | 2380 | 2300 | 2200 | 3430 | 1960 | 2440 | **630** |
| Mean | 4 | 2006.33 | 1961.33 | 2105.33 | 2502.33 | 1409.00 | 2074.67 | **951.00** |
| | 5 | 1949.33 | 1786.67 | 1751.33 | 2045.33 | 1505.33 | 1688.33 | **1395.33** |
| | 6 | 1650.00 | 1643.33 | 1569.00 | 2012.00 | 1443.00 | 1728.00 | **479.67** |
| Std | 4 | 4.44E+02 | 4.50E+02 | 4.43E+02 | 8.18E+02 | 2.56E+02 | 4.38E+02 | **1.46E+02** |
| | 5 | 4.19E+02 | 3.47E+02 | 3.33E+02 | 5.28E+02 | 3.05E+02 | 3.05E+02 | **2.20E+02** |
| | 6 | 3.28E+02 | 3.01E+02 | 3.07E+02 | 4.60E+02 | 2.61E+02 | 2.75E+02 | **7.88E+01** |
| p | 4 | **3.65E-11** | **5.43E-11** | **2.98E-11** | **3.48E-11** | **2.41E-09** | **3.31E-11** | ~ |
| | 5 | **2.01E-07** | **3.31E-06** | **3.94E-05** | **7.28E-07** | 3.29E-01 | **6.53E-05** | ~ |
| | 6 | **2.97E-11** | **2.97E-11** | **2.95E-11** | **2.98E-11** | **2.95E-11** | **2.94E-11** | ~ |

**Table 8**
Statistical Results for Indicator $T_{init}$ in the Special complex passageway.

| | Map | CDRT-RRT* | CloudRRT* | InformedRRT* | RRT* | RRT*smart | TQRRT* | HAURRT*smart |
|---|---|---|---|---|---|---|---|---|
| Best | 4 | 4.56 | 4.27 | 5.69 | 4.85 | 3.13 | 4.83 | **2.78** |
| | 5 | 4.40 | 4.02 | 4.37 | 4.74 | **2.81** | 3.06 | 3.18 |
| | 6 | 3.90 | 4.02 | 3.93 | 4.96 | 2.89 | 4.49 | **0.95** |
| Worst | 4 | 12.37 | 11.62 | 13.10 | 17.53 | 6.61 | 11.36 | **5.16** |
| | 5 | 9.70 | 9.43 | 9.12 | 13.14 | 6.40 | 7.68 | **6.34** |
| | 6 | 8.42 | 8.42 | 7.93 | 12.20 | 6.45 | 8.97 | **2.31** |
| Mean | 4 | 7.20 | 7.17 | 7.82 | 9.18 | 4.51 | 7.61 | **3.51** |
| | 5 | 6.98 | 6.49 | 6.41 | 7.35 | **4.69** | 6.12 | 5.04 |
| | 6 | 5.85 | 5.89 | 5.74 | 7.24 | 4.64 | 6.27 | **1.72** |
| Std | 4 | 1.65E+00 | 1.70E+00 | 1.69E+00 | 3.07E+00 | 7.59E-01 | 1.61E+00 | **5.72E-01** |
| | 5 | 1.51E+00 | 1.25E+00 | 1.20E+00 | 1.90E+00 | 8.95E-01 | 1.09E+00 | **7.95E-01** |
| | 6 | 1.17E+00 | 1.12E+00 | 1.13E+00 | 1.67E+00 | 8.61E-01 | 1.01E+00 | **2.97E-01** |
| p | 4 | **5.49E-11** | **5.49E-11** | **3.02E-11** | **3.69E-11** | **1.11E-06** | **3.69E-11** | ~ |
| | 5 | **3.52E-07** | **3.32E-06** | **6.74E-06** | **4.11E-07** | 1.33E-01 | **3.37E-05** | ~ |
| | 6 | **3.02E-11** | **3.02E-11** | **3.02E-11** | **3.02E-11** | **3.02E-11** | **3.02E-11** | ~ |

and TQRRT* show areas with absent random tree branches, while RRT* and RRT*smart miss some obstacle corners. HAURRT*smart thoroughly explores all areas in the complex maps.

In the reality-simulating environment, Fig. 26 shows the coverage efficiency of eight algorithms over iterations. HAURRT*smart also maintains low node redundancy, outperforming the others.

### 4.5. Dynamic environment

In Sections 4.1– 4.4, we noted that HAURRT*smart offers high efficiency, low redundancy, and adaptability to various environments, maintaining strong sampling balance and efficient coverage of the search space. This efficiency persists in dynamic environments. The

algorithm employs the Halton sequence for sampling points, with each point linked to a label $U(n)$. When a point is chosen, $U(n)$ changes, allowing us to extend HAURRT*smart to dynamic settings.

Due to the fact that HAURRT*smart has not been specifically adapted or refined for dynamic environments, quantitative analysis results for this section are not provided. The primary innovation of HAURRT*smart resides in its utilization of pseudo-random sequences to deliberately modify the distribution of path nodes. As a result, most of the nodes in the random trees generated by HAURRT*smart can be indexed using these pseudo-random sequences. Dynamic experiments indicate that if the distribution of path nodes can be indexed and controlled, certain advantages of the algorithm—such as its low redundancy characteristic—can be effectively applied to dynamic environments. The performance of HAURRT*smart in dynamic contexts is presented as supplementary material. Fig. 27 shows video screenshots at three timestamps (0:48, 1:23, and 1:53) from this material.

In Fig. 27 (A), we use Map1 for testing. When the node count reaches 500, 700, and 900, we delete random trees in the right half and generate new sampling nodes on the remaining trees, simulating damage due to terrain changes. In this scenario, HAURRT*smart continues to search and maintains sampling balance and low redundancy. In Fig. 27 (B), we tested HAURRT*smart on Maps 2, 3, and 4, with consistent results. Fig. 27 (C) introduces dynamic circular obstacles in Map2 and Map3, and HAURRT*smart still provides near-optimal paths quickly while maintaining the advantages of high speed, balance, and low redundancy.

## 5. Conclusion

This paper introduces HAURRT*smart, a novel path planning algorithm that enhances traditional RRT methods for complex environments. It utilizes Halton sequences for more uniform node sampling, significantly reducing redundancy and improving efficiency. The algorithm features an adaptive goal bias strategy that adjusts sampling based on environmental complexity, facilitating effective exploration toward the goal. HAURRT*smart also implements a node chain expansion strategy for compact tree growth and incorporates a path optimization technique to enhance output smoothness and feasibility. These improvements enhance the algorithm's performance in static and dynamic environments. However, HAURRT*smart has limitations; First, because the pseudo-random sequence's density automatically adjusts the branch-expansion step size, initial expansion difficulties may arise in scenarios with extremely dense obstacles. Second, the algorithm fails to fully leverage saved paths during iteration for targeted precision optimization, making it ill-suited for high-accuracy applications. Finally, in dynamic settings, deleted random tree branches are removed without fully utilizing branch structure information beforehand.

Experimental results show that HAURRT*smart excels in complex environments, surpassing existing algorithms in path search efficiency, node utilization, and path quality. In simpler settings, it quickly finds feasible paths, while in intricate channels and mazes, it effectively balances exploration and optimization. Overall, HAURRT*smart integrates advanced strategies for efficient path planning, offering significant potential for real-world applications and laying a foundation for further research in path planning. In the future, we will enhance the HAURRT*smart for high-dimensional dynamic environments and introduce new local search strategies to improve optimization accuracy while retaining fast search capabilities.

## CRediT authorship contribution statement

**Peidong He:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation; **Gang Hu:** Writing – review & editing, Writing – original draft, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization; **Essam H. Houssein:** Writing – review & editing, Writing – original draft, Visualization, Supervision, Resources, Methodology, Investigation, Formal analysis, Conceptualization; **Guo Wei:** Writing – review & editing, Writing – original draft, Supervision, Resources, Methodology, Investigation, Formal analysis, Conceptualization.

## Data availability

Data will be made available on request.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## References

[1] F.H. Ajeil, I.K. Ibraheem, A.T. Azar, A.J. Humaidi, Grid-Based mobile robot path planning using aging-Based ant colony optimization algorithm in static and dynamic environments, Sensors 20 (7) (2020) 1880. https://doi.org/10.3390/s20071880

[2] B. Liao, F. Wan, Y. Hua, R. Ma, S. Zhu, X. Qing, F-RRT*: An improved path planning algorithm with improved initial solution and convergence rate, Expert Syst. Appl. 184 (2021) 115457. https://doi.org/10.1016/j.eswa.2021.115457

[3] L. Jiang, C. Zhang, Z. Hu, R. He, H. Xiao, W. Xu, J. Meng, Safe path planning for autonomous vehicles with real-Time observation based localization uncertainty prediction, Expert Syst. Appl. 281 (2025) 127596. https://doi.org/10.1016/j.eswa.2025.127596

[4] R. Levy, J. Haddad, Cooperative path and trajectory planning for autonomous vehicles on roads without lanes: a laboratory experimental demonstration, Transp. Res. Part C: Emerg. Technol. 144 (2022) 103813. https://doi.org/10.1016/j.trc.2022.103813

[5] W. Zhang, T. Sun, Y. Li, C. He, X. Xiu, Z. Miao, Optimal motion planning and navigation for nonholonomic agricultural robots in multi-Constraint and multi-Task environments, Comput. Electron. Agric. 238 (2025) 110822. https://doi.org/10.1016/j.compag.2025.110822

[6] B. Zhou, J. Zhao, T. Zhang, R. Xia, H. Wang, J. Wang, L. Gao, Robotic aero-Engine pipe grasping posture and motion planning method in multi-Stage processing based on multi-Objective optimization, J. Manuf. Syst. 81 (2025) 117–143. https://doi.org/10.1016/j.jmsy.2025.05.011

[7] T.S. Lakhwani, Y. Sinjana, A metaheuristic approach for optimizing drone routing in healthcare supply chains, Supply Chain Anal. (2025) 100153. https://doi.org/10.1016/j.sca.2025.100153

[8] G. Hu, P. He, G. Wei, Distributed large-Scale joint non-Uniform UAV formation path planning based on global optimal guidance, Appl. Math. Model. 148 (2025) 116239. https://doi.org/10.1016/j.apm.2025.116239

[9] M. Mohammadi, S.M.R. Nazemosadat, D. Fazel, Y. Bazargan Lari, An integrated approach for structural modeling, modal analysis, and aerodynamic evaluation of an electric vehicle body shell using finite element method and computational fluid dynamics, Mater. Today Commun. 45 (2025) 112331. https://doi.org/10.1016/j.mtcomm.2025.112331

[10] Y.-m. Liang, H.-y. Zhao, CCPF-RRT*: An improved path planning algorithm with consideration of congestion, Expert Syst. Appl. 228 (2023) 120403. https://doi.org/10.1016/j.eswa.2023.120403

[11] S. Bouraine, Y. Bellalia, I. Chaabeni, D. Naceur, When robots learn from nature: GLWOA-RRT*, a nature-Inspired motion planning approach, Swarm Evol. Comput. 98 (2025) 102062. https://doi.org/10.1016/j.swevo.2025.102062

[12] L. Yang, J. Qi, J. Xiao, X. Yong, A literature review of UAV 3D path planning, in: Proceeding of the 11Th World Congress on Intelligent Control and Automation, IEEE, Shenyang, China, 2014, pp. 2376–2381. https://doi.org/10.1109/WCICA.2014.7053093

[13] X. Cui, C. Wang, Y. Xiong, L. Mei, S. Wu, More quickly-RRT*: improved quick rapidly-Exploring random tree star algorithm based on optimized sampling point with better initial solution and convergence rate, Eng. Appl. Artif. Intell. 133 (2024) 108246. https://doi.org/10.1016/j.engappai.2024.108246

[14] S.M. Langbak, C. Schou, K.D. Hansen, Multi-Autonomous mobile robot traffic management based on layered costmaps and a modified Dijkstra's algorithm, Procedia Comput. Sci. 232 (2024) 53–63. https://doi.org/10.1016/j.procs.2024.01.006

[15] B. Zhang, J. Cao, X. Li, W. Chen, X. Zheng, Y. Zhang, Y. Song, Minimum dose walking path planning in a nuclear radiation environment based on a modified a* algorithm, Ann. Nucl. Energy 206 (2024) 110629. https://doi.org/10.1016/j.anucene.2024.110629

[16] M. Dakulović, S. Horvatić, I. Petrović, Complete coverage d* algorithm for path planning of a floor-Cleaning mobile robot, IFAC Proc. Vols. 44 (1) (2011) 5950–5955. https://doi.org/10.3182/20110828-6-IT-1002.03400

[17] I. Sung, B. Choi, P. Nielsen, On the training of a neural network for online path planning with offline path planning algorithms, Int. J. Inf. Manag. 57 (2021) 102142. https://doi.org/10.1016/j.ijinfomgt.2020.102142

[18] J. Fan, X. Chen, X. Liang, UAV Trajectory planning based on bi-Directional APF-RRT* algorithm with goal-Biased, Expert Syst. Appl. 213 (2023) 119137. https://doi.org/10.1016/j.eswa.2022.119137
[19] C. Cheng, Q. Sha, B. He, G. Li, Path planning and obstacle avoidance for AUV: a review, Ocean Eng. 235 (2021) 109355. https://doi.org/10.1016/j.oceaneng.2021.109355
[20] J. Cui, L. Wu, X. Huang, D. Xu, C. Liu, W. Xiao, Multi-Strategy adaptable ant colony optimization algorithm and its application in robot path planning, Knowl. Based Syst. 288 (2024) 111459. https://doi.org/10.1016/j.knosys.2024.111459
[21] S. Lin, A. Liu, J. Wang, X. Kong, An intelligence-Based hybrid PSO-SA for mobile robot path planning in warehouse, J. Comput. Sci. 67 (2023) 101938. https://doi.org/10.1016/j.jocs.2022.101938
[22] X. Yu, N. Jiang, X. Wang, M. Li, A hybrid algorithm based on grey wolf optimizer and differential evolution for UAV path planning, Expert Syst. Appl. 215 (2023) 119327. https://doi.org/10.1016/j.eswa.2022.119327
[23] R. Sarkar, D. Barman, N. Chowdhury, Domain knowledge based genetic algorithms for mobile robot path planning having single and multiple targets, J. King Saud Univ. - Comput. Inf. Sci. 34 (7) (2022) 4269–4283. https://doi.org/10.1016/j.jksuci.2020.10.010
[24] G. Li, C. Liu, L. Wu, W. Xiao, A mixing algorithm of ACO and ABC for solving path planning of mobile robot, Appl. Soft Comput. 148 (2023) 110868. https://doi.org/10.1016/j.asoc.2023.110868
[25] G. Hu, J. Zhong, G. Wei, Sachba_PDN: modified honey badger algorithm with multi-Strategy for UAV path planning, Expert Syst. Appl. 223 (2023) 119941. https://doi.org/10.1016/j.eswa.2023.119941
[26] Y. Cui, W. Hu, A. Rahmani, Multi-Robot path planning using learning-Based artificial bee colony algorithm, Eng. Appl. Artif. Intell. 129 (2024) 107579. https://doi.org/10.1016/j.engappai.2023.107579
[27] R. Akay, M.Y. Yildirim, Multi-Strategy and self-Adaptive differential sine–Cosine algorithm for multi-Robot path planning, Expert Syst. Appl. 232 (2023) 120849. https://doi.org/10.1016/j.eswa.2023.120849
[28] G. Hu, F. Huang, A. Seyyedabbasi, G. Wei, Enhanced multi-Strategy bottlenose dolphin optimizer for UAVs path planning, Appl. Math. Model. 130 (2024) 243–271. https://doi.org/10.1016/j.apm.2024.03.001
[29] M. Mohammadi, Y. Bazargan Lari, K. Bazargan Lari, Centipede optimization algorithm: best-of-L local search with contractive consensus for constrained engineering optimization, Neurocomputing 659 (2026) 131774. https://doi.org/10.1016/j.neucom.2025.131774
[30] B. Sahu, P.K. Das, R. Kumar, A modified cuckoo search algorithm implemented with SCA and PSO for multi-Robot cooperation and path planning, Cogn. Syst. Res. 79 (2023) 24–42. https://doi.org/10.1016/j.cogsys.2023.01.005
[31] E. Zakeri, S.A. Moezi, Y. Bazargan-Lari, A. Zare, Multi-Tracker optimization algorithm: a general algorithm for solving engineering optimization problems, Iranian J. Sci. Technol. Trans. Mech. Eng. 41 (4) (2017) 315–341. https://doi.org/10.1007/s40997-016-0066-9
[32] K. Yan, D. Liu, Z. Liu, J. Tan, An inspection path planning approach based on hierarchical reinforcement learning, J. Manuf. Syst. 85 (2026) 34–50. https://doi.org/10.1016/j.jmsy.2025.12.027
[33] H. Fang, W.C. Tam, R. Lovreglio, M.I.S. Emon, M.X. Huang, Development of a tenability-Based path planning model for building fire evacuations using reinforcement learning, J. Build. Eng. 118 (2026) 115132. https://doi.org/10.1016/j.jobe.2025.115132
[34] H. Xu, J. Dai, M. Chen, Z. Guo, X. Jiang, Z. Li, L. Zhou, T. Yang, W. Pang, Deep learning-Based optimal evacuation path planning for movie theater auditoriums, J. Build. Eng. 108 (2025) 112822. https://doi.org/10.1016/j.jobe.2025.112822
[35] R. Van Essen, E. Van Henten, G. Kootstra, UAV-Based path planning for efficient localization of non-Uniformly distributed weeds using prior knowledge: a reinforcement-Learning approach, Comput. Electron. Agric. 237 (2025) 110651. https://doi.org/10.1016/j.compag.2025.110651
[36] W. Song, Z. Chen, M. Sun, Y. Wang, Q. Sun, A COLREGs-based path-Planning method for collision avoidance considering path cost through reinforcement learning, Ocean Eng. 325 (2025) 120746. https://doi.org/10.1016/j.oceaneng.2025.120746
[37] L. Wang, S. Zheng, S. Tai, H. Liu, T. Yue, UAV Air combat autonomous trajectory planning method based on robust adversarial reinforcement learning, Aerosp. Sci. Technol. 153 (2024) 109402. https://doi.org/10.1016/j.ast.2024.109402
[38] B. Wang, X. Gao, T. Xie, An evolutionary multi-Agent reinforcement learning algorithm for multi-UAV air combat, Knowl. Based Syst. 299 (2024) 112000. https://doi.org/10.1016/j.knosys.2024.112000
[39] B. Zhao, M. Huo, Z. Li, Z. Yu, N. Qi, Graph-Based multi-Agent reinforcement learning for large-Scale UAVs swarm system control, Aerosp. Sci. Technol. 150 (2024) 109166. https://doi.org/10.1016/j.ast.2024.109166
[40] Y. Song, G. Chen, Y. Chen, J. Gao, Reinforcement learning-Based fast-Dual-Tree RRT path planning for unmanned underwater vehicles, Ocean Eng. 342 (2025) 122937. https://doi.org/10.1016/j.oceaneng.2025.122937
[41] R. Jin, Path planning and control of building robots based on reinforcement learning algorithm in intelligent construction, Procedia Comput. Sci. 261 (2025) 637–646. https://doi.org/10.1016/j.procs.2025.04.255
[42] Z. Sun, B. Xia, P. Xie, X. Li, J. Wang, NAMR-RRT: Neural adaptive motion planning for mobile robots in dynamic environments, IEEE Trans. Autom. Sci. Eng. 22 (2025) 13087–13100. https://doi.org/10.1109/TASE.2025.3551464
[43] D. Ma, S. Hao, W. Ma, H. Zheng, X. Xu, An optimal control-Based path planning method for unmanned surface vehicles in complex environments, Ocean Eng. 245 (2022) 110532. https://doi.org/10.1016/j.oceaneng.2022.110532
[44] X.-w. Wang, H.-j. Peng, J. Liu, X.-z. Dong, X.-d. Zhao, C. Lu, Optimal control based coordinated taxiing path planning and tracking for multiple carrier aircraft on flight deck, Defence Technol. 18 (2) (2022) 238–248. https://doi.org/10.1016/j.dt.2020.11.013
[45] K. Bergman, O. Ljungqvist, D. Axehill, Improved path planning by tightly combining lattice-Based path planning and optimal control, IEEE Trans. Intell. Veh. 6 (1) (2021) 57–66. https://doi.org/10.1109/TIV.2020.2991951
[46] Y. Farid, R.M. Jungers, Binary combinatorial optimization-Based path planning and optimal transition control in piecewise linear neural abstraction domain, Neurocomputing 669 (2026) 132504. https://doi.org/10.1016/j.neucom.2025.132504
[47] H. Ren, S. Chen, L. Yang, Y. Zhao, Optimal path planning and speed control integration strategy for UGVs in static and dynamic environments, IEEE Trans. Veh. Technol. 69 (10) (2020) 10619–10629. https://doi.org/10.1109/TVT.2020.3015582
[48] L. Zhou, Z. Li, Y. Li, S. Bai, Parallel MPPI with gradient-Velocity modulated SDF cost for high-Performance real-Time dynamic obstacle avoidance by robot manipulators, IEEE Trans. Rob. 41 (2025) 5149–5168. https://doi.org/10.1109/TRO.2025.3600125
[49] L. Wang, D. Ye, Y. Xiao, X. Kong, Trajectory planning for satellite cluster reconfigurations with sequential convex programming method, Aerosp. Sci. Technol. 136 (2023) 108216. https://doi.org/10.1016/j.ast.2023.108216
[50] J. Guo, Y. Li, B. Huang, L. Ding, H. Gao, M. Zhong, An online optimization escape entrapment strategy for planetary rovers based on bayesian optimization, J. Field Rob. 41 (8) (2024) 2518–2529. https://doi.org/10.1002/rob.22361
[51] H. Qi, L. Ding, M. Zheng, L. Huang, H. Gao, G. Liu, Z. Deng, Variable wheelbase control of wheeled mobile robots with worm-Inspired creeping gait strategy, IEEE Trans. Rob. 40 (2024) 3271–3289. https://doi.org/10.1109/TRO.2024.3400947
[52] G. Du, Y. Zou, X. Zhang, Z. Li, Q. Liu, Hierarchical path planning and motion control framework using adaptive scale based bidirectional search and heuristic learning based predictive control, IEEE Trans. Veh. Technol. 74 (6) (2025) 8647–8663. https://doi.org/10.1109/TVT.2025.3532643
[53] Z. Sheng, T. Song, J. Song, Y. Liu, P. Ren, Bidirectional rapidly exploring random tree path planning algorithm based on adaptive strategies and artificial potential fields, Eng. Appl. Artif. Intell. 148 (2025) 110393. https://doi.org/10.1016/j.engappai.2025.110393
[54] J. Chang, A. Wang, X. Fan, P. Su, Improved RRT-D*: algorithm fusion for optimal path planning, in: 2024 International Conference on Intelligent Robotics and Automatic Control (IRAC), IEEE, Guangzhou, China, 2024, pp. 259–266. https://doi.org/10.1109/IRAC63143.2024.10871681
[55] L. Liu, X. Wang, X. Yang, H. Liu, J. Li, P. Wang, Path planning techniques for mobile robots: review and prospect, Expert Syst. Appl. 227 (2023) 120254. https://doi.org/10.1016/j.eswa.2023.120254
[56] S. Yu, J. Chen, G. Liu, X. Tong, Y. Sun, SOF-RRT*: An improved path planning algorithm using spatial offset sampling, Eng. Appl. Artif. Intell. 126 (2023) 106875. https://doi.org/10.1016/j.engappai.2023.106875
[57] J. Wang, E. Zheng, Path planning of mobile robot based on the improved RRT-connect algorithm, in: 2024 3Rd International Conference on Service Robotics (ICoSR), IEEE, Hangzhou, China, 2024, pp. 34–38. https://doi.org/10.1109/ICoSR63848.2024.00018
[58] Y. Wu, R. Guo, Z. Yan, G. Cui, K. Yang, K. Zhuo, Robotic arm motion planning for obstacle avoidance based on the CMN-RRT * method, in: 2023 IEEE International Conference on Robotics and Biomimetics (ROBIO), IEEE, Koh Samui, Thailand, 2023, pp. 1–6. https://doi.org/10.1109/ROBIO58561.2023.10354552
[59] S. Ganesan, A hybrid sampling-Based RRT* path planning algorithm for autonomous mobile robot navigation, Expert Syst. Appl. (2024). https://doi.org/10.1016/j.eswa.2024.125206
[60] J.D. Gammell, S.S. Srinivasa, T.D. Barfoot, Informed RRT*: optimal sampling-Based path planning focused via direct sampling of an admissible ellipsoidal heuristic, in: 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems, IEEE, Chicago, IL, USA, 2014, pp. 2997–3004. https://doi.org/10.1109/IROS.2014.6942976
[61] F. Yu, Y. Chen, Cyl-IRRT*: homotopy optimal 3D path planning for AUVs by biasing the sampling into a cylindrical informed subset, IEEE Trans. Ind. Electron. 70 (4) (2023) 3985–3994. https://doi.org/10.1109/TIE.2022.3177801
[62] F. Islam, J. Nasir, U. Malik, Y. Ayaz, O. Hasan, RRT∗-Smart: rapid convergence implementation of RRT∗ towards optimal solution, in: 2012 IEEE International Conference on Mechatronics and Automation, IEEE, Chengdu, China, 2012, pp. 1651–1656. https://doi.org/10.1109/ICMA.2012.6284384
[63] X. Cao, H. Shen, J. Wang, Y. Zhao, K. Wang, X. Fei, AE-RRT*: Adaptive ellipse-Based optimal path planning, in: 2024 36Th Chinese Control and Decision Conference (CCDC), IEEE, Xi'an, China, 2024, pp. 1501–1506. https://doi.org/10.1109/CCDC62350.2024.10588139
[64] K. Mittal, S. Gupta, Minimum-Time motion-Planning of AUVs under spatially varying ocean currents, in: Oceans 2019 Mts/Ieee Seattle, IEEE, Seattle, WA, USA, 2019, pp. 1–5. https://doi.org/10.23919/OCEANS40490.2019.8962873
[65] Y. Li, W. Wei, Y. Gao, D. Wang, Z. Fan, PQ-RRT*: An improved path planning algorithm for mobile robots, Expert Syst. Appl. 152 (2020) 113425. https://doi.org/10.1016/j.eswa.2020.113425
[66] W. Jin, X. Ma, Y. Li, Research on the AGV global path planning algorithm based on FI-RRT*, in: 2023 IEEE 5Th International Conference on Power, Intelligent Computing and Systems (ICPICS), IEEE, Shenyang, China, 2023, pp. 104–108. https://doi.org/10.1109/ICPICS58376.2023.10235429
[67] W. Zhang, L. Shan, L. Chang, Y. Dai, SVF-RRT*: A stream-Based VF-RRT* for USVs path planning considering ocean currents, IEEE Rob. Autom. Lett. 8 (4) (2023) 2413–2420. https://doi.org/10.1109/LRA.2023.3245409
[68] T. Huang, K. Fan, W. Sun, Density gradient-RRT: an improved rapidly exploring random tree algorithm for UAV path planning, Expert Syst. Appl. 252 (2024) 124121. https://doi.org/10.1016/j.eswa.2024.124121

[69] H. Zhong, M. Cong, M. Wang, Y. Du, D. Liu, HB-RRT:A path planning algorithm for mobile robots using halton sequence-Based rapidly-Exploring random tree, Eng. Appl. Artif. Intell. 133 (2024) 108362. https://doi.org/10.1016/j.engappai.2024.108362
[70] A. Tahmasbi, M. Faghfoorian, S. Khodaygan, A. Bera, Zonal RL-RRT: integrated RL-RRT path planning with collision probability and zone connectivity (2024). https://doi.org/10.48550/arXiv.2410.24205
[71] D. Zhang, H. Duan, Variational resolution probability maps for multi-UAV search tasks with changeable searching behaviors, Aerosp. Sci. Technol. 155 (2024) 109669. https://doi.org/10.1016/j.ast.2024.109669
[72] B. An, J. Kim, F.C. Park, An adaptive stepsize RRT planning algorithm for open-Chain robots, IEEE Rob. Autom. Lett. 3 (1) (2018) 312–319. https://doi.org/10.1109/LRA.2017.2745542
[73] L. Jiang, S. Liu, Y. Cui, H. Jiang, Path planning for robotic manipulator in complex multi-Obstacle environment based on improved_RRT, IEEE/ASME Trans. Mechatron. 27 (6) (2022) 4774–4785. https://doi.org/10.1109/TMECH.2022.3165845
[74] Z. Feng, L. Zhou, J. Qi, S. Hong, DBVS-APF-RRT*: A global path planning algorithm with ultra-High speed generation of initial paths and high optimal path quality, Expert Syst. Appl. 249 (2024) 123571. https://doi.org/10.1016/j.eswa.2024.123571
[75] C. Liu, F. Xiao, Y. Ma, H. Chen, Y. Wu, Z. Li, L. Guo, An enhanced RRT* algorithm with biased sampling and dynamic stepsize strategy for ship route planning in the high-Risk areas, Ocean Eng. 332 (2025) 121466. https://doi.org/10.1016/j.oceaneng.2025.121466
[76] M.F. Aslan, A. Durdu, K. Sabanci, Goal distance-Based UAV path planning approach, path optimization and learning-Based path estimation: GDRRT*, PSO-GDRRT* and biLSTM-PSO-GDRRT*, Appl. Soft Comput. 137 (2023) 110156. https://doi.org/10.1016/j.asoc.2023.110156
[77] S. Guo, J. Gong, H. Shen, L. Yuan, W. Wei, Y. Long, DBVSB-P-RRT*: A path planning algorithm for mobile robot with high environmental adaptability and ultra-High speed planning, Expert Syst. Appl. 266 (2025) 126123. https://doi.org/10.1016/j.eswa.2024.126123
[78] C. Jiang, Z. Hu, Z.P. Mourelatos, D. Gorsich, P. Jayakumar, Y. Fu, M. Majcher, R2-RRT*: Reliability-based robust mission planning of off-Road autonomous ground vehicle under uncertain terrain environment, IEEE Trans. Autom. Sci. Eng. 19 (2) (2022) 1030–1046. https://doi.org/10.1109/TASE.2021.3050762
[79] Y. Ning, T. Li, C. Yao, W. Du, Y. Zhang, HMS-RRT: A novel hybrid multi-Strategy rapidly-Exploring random tree algorithm for multi-Robot collaborative exploration in unknown environments, Expert Syst. Appl. 247 (2024) 123238. https://doi.org/10.1016/j.eswa.2024.123238
[80] D. Kim, J. Lee, S.-e. Yoon, Cloud RRT*: sampling cloud based RRT*, in: 2014 IEEE International Conference on Robotics and Automation (ICRA), IEEE, Hong Kong, China, 2014, pp. 2519–2526. https://doi.org/10.1109/ICRA.2014.6907211
[81] J. Liu, M. Fu, W. Zhang, B. Chen, U. Sychou, A. Belotserkovsky, CDRT-RRT*: Real-time rapidly exploring random tree star based on convex dissection, Expert Syst. Appl. 268 (2025) 126291. https://doi.org/10.1016/j.eswa.2024.126291
[82] D. Uwacu, A. Yammanuru, K. Nallamotu, V. Chalasani, M. Morales, N.M. Amato, HAS-RRT: RRT-Based motion planning using topological guidance, IEEE Rob. Autom. Lett. 10 (6) (2025) 6223–6230. https://doi.org/10.1109/LRA.2025.3560878
[83] F. Meng, L. Chen, H. Ma, J. Wang, M.Q.H. Meng, NR-RRT: Neural risk-Aware near-Optimal path planning in uncertain nonconvex environments, IEEE Trans. Autom. Sci. Eng. 21 (1) (2024) 135–146. https://doi.org/10.1109/TASE.2022.3215562
[84] Z. Huang, Y. Gao, J. Guo, C. Qian, Q. Chen, An adaptive bidirectional quick optimal rapidly-Exploring random tree algorithm for path planning, Eng. Appl. Artif. Intell. 135 (2024) 108776. https://doi.org/10.1016/j.engappai.2024.108776
[85] J. Fan, X. Chen, Y. Wang, X. Chen, UAV Trajectory planning in cluttered environments based on PF-RRT* algorithm with goal-Biased strategy, Eng. Appl. Artif. Intell. 114 (2022) 105182. https://doi.org/10.1016/j.engappai.2022.105182
[86] Q. Chen, M. Wang, An improved RRT* algorithm for mobile robots path planning, in: 2022 China Automation Congress (CAC), IEEE, Xiamen, China, 2022, pp. 4334–4339. https://doi.org/10.1109/CAC57257.2022.10055894
[87] Y. Zhang, H. Wang, M. Yin, J. Wang, C. Hua, Bi-AM-RRT*: a fast and efficient sampling-Based motion planning algorithm in dynamic environments, IEEE Trans. Intell. Veh. 9 (1) (2024) 1282–1293. https://doi.org/10.1109/TIV.2023.3307283
[88] H. Huang, Y. Shang, X. Liu, X. Liu, P. Qi, An improved bi-RRT*-Based path planning algorithm with adaptive search strategy assignment mechanism for ultra-Low-Altitude penetration of fixed-Wing aircraft, Aerosp. Sci. Technol. (2024) 109363. https://doi.org/10.1016/j.ast.2024.109363
[89] Y. Li, R. Juan, Y. Zhou, T. Wang, Z. Li, W. Guo, Z. Gao, AD-RRT*: An RRT*-Based global path planning approach for underwater gliders with alpha shapes and DBSCAN, Expert Syst. Appl. 291 (2025) 128219. https://doi.org/10.1016/j.eswa.2025.128219
[90] S. Yao, X. Wang, S. Huang, R. Xu, Y. Zhao, Unmanned aerial vehicle takeoff point search algorithm with information sharing strategy of random trees for multi-Area coverage task, Appl. Soft Comput. 174 (2025) 112970. https://doi.org/10.1016/j.asoc.2025.112970
[91] L. Zhang, D. Yang, B. Niu, H. Yang, Q. Huang, L. Jiang, H. Liu, Smooth path planning and dynamic contact force regulation for robotic ultrasound scanning, IEEE Rob. Autom. Lett. 10 (10) (2025) 10570–10577. https://doi.org/10.1109/LRA.2025.3604746
[92] B. Wang, D. Ju, Q. Li, C. Feng, SS-RRT*: A safe and smoothing path planner for mobile robot in static environment, in: 2022 China Automation Congress (CAC), IEEE, Xiamen, China, 2022, pp. 3677–3682. https://doi.org/10.1109/CAC57257.2022.10054840
[93] Y. Wang, W. Jiang, Z. Luo, L. Yang, J. Li, H. Lu, Path optimization of a flexible robot with a spatial compressed and a direction-Guided exploring method, Eng. Appl. Artif. Intell. 160 (2025) 111846. https://doi.org/10.1016/j.engappai.2025.111846
[94] T. Xu, J. Ma, P. Qin, Q. Hu, An improved RRT* algorithm based on adaptive informed sample strategy for coastal ship path planning, Ocean Eng. 333 (2025) 121511. https://doi.org/10.1016/j.oceaneng.2025.121511
[95] J. Yu, C. Chen, A. Arab, J. Yi, X. Pei, X. Guo, RDT-RRT: Real-time double-Tree rapidly-Exploring random tree path planning for autonomous vehicles, Expert Syst. Appl. 240 (2024) 122510. https://doi.org/10.1016/j.eswa.2023.122510
[96] B. Lindqvist, A. Patel, K. Löfgren, G. Nikolakopoulos, A tree-Based next-Best-Trajectory method for 3-d UAV exploration, IEEE Trans. Rob. 40 (2024) 3496–3513. https://doi.org/10.1109/TRO.2024.3422052
[97] S. Zhang, R. Cui, W. Yan, Y. Li, RA-RRTV*: Risk-averse RRT* with local vine expansion for path planning in narrow passages under localization uncertainty, IEEE Rob. Autom. Lett. 10 (2) (2025) 2072–2079. https://doi.org/10.1109/LRA.2025.3528675
[98] Z. Huang, D. Yang, Z. Cheng, X. Jiang, Towards autonomous premium tea harvesting: a high-Efficiency and high-Quality path planning based on optimized IABFMT* algorithm in unstructured canopies, Smart Agric. Technol. (2025) 101317. https://doi.org/10.1016/j.atech.2025.101317
[99] H. Zhong, Y. Du, D. Liu, M. Wang, M. Cong, X. Tian, A fruit fly-Inspired path planning algorithm for unmanned aerial vehicle in underground environments based on low-Discrepancy sequences, Eng. Appl. Artif. Intell. 156 (2025) 111250. https://doi.org/10.1016/j.engappai.2025.111250
[100] L. Kocis, W.J. Whiten, Computational investigations of low-Discrepancy sequences, ACM Trans. Math. Softw. 23 (2) (1997) 266–294. https://doi.org/10.1145/264029.264064
[101] S. Karaman, E. Frazzoli, Sampling-Based algorithms for optimal motion planning, Int. J. Rob. Res. 30 (7) (2011) 846–894. https://doi.org/10.1177/0278364911406761